



华南理工大学
South China University of Technology

高等教育自学考试

毕业论文（设计）

基于 Vue3 和 Spring 开发基础企业管理系统

办学单位：_____华南理工大学_____

届 • 专业：_____2025 届计算机科学与技术_____

学 生：_____廖凯 (030519202188)_____

指导教师：_____陈翠松 副教授_____

提交日期：_____2025_____年_____03_____月_____30_____日_____

学位论文原创性声明

本人郑重声明： 所呈交的学位论文是本人在导师指导下进行的研究工作所取得的
研究成果，除了文中已经注明引用的内容外，论文中不包含其他个人或集体已经发表或
撰写过的研究成果。对本文的研究作出重要贡献的个人和集体，均已在文中以明确方式
标明。本声明的法律后果完全由本人承担。

作者签名： _____

日期： _____ 年 _____ 月 _____ 日

学位论文授权使用声明

本人完全了解华南理工大学有关保留和使用学位论文的规定，即学位论文所涉及的
知识产权属于华南理工大学。学校有权保留并向国家有关部门或机构送交论文的复印件
和电子版，允许学位论文被查阅和借阅；学校可以公布学位论文的全部或部分内容，可
以采用影印、缩印或其它复制手段保存、汇编学位论文。（保密的学位论文在解密后遵
守此规定）

作者签名： _____ 导师签名： _____

日期： _____ 年 _____ 月 _____ 日

摘 要

随着互联网技术的持续普及和信息技术的迅猛发展，企业管理系统在现代企业运营中的重要性日益凸显。企业管理系统致力于通过信息化手段提升管理效率、促进团队协作、优化资源配置、提高决策质量，进而增强企业的市场竞争力。但现行的企业管理系统在功能完备性、可扩展性以及用户体验等方面仍存在缺陷，难以适应企业日益增长的复杂和多样化管理需求。

本项目的设计目标是开发一个基于 Vue 3 和 Spring Boot 的企业级管理系统模板，采用现代化的前后端技术架构，旨在提供高效、优质的开发流程和卓越的用户体验，同时提供管理系统的基础核心功能，以满足企业多样化的信息管理需求。为了提升企业信息管理系统的用户体验，系统支持自适应布局和主题个性化功能。系统界面应支持响应式设计，可以根据客户端的设备类型和屏幕尺寸自动调整布局。同时系统还支持深色和浅色主题等主题化功能。在系统开发方面，采用组件化和 Hooks 化的开发模式提高了代码复用性和可维护性。TypeScript 类型检查以及代码提示功能增强了代码质量和开发效率。此外，系统还采用了多级缓存策略，包括 Caffeine 本地缓存和 Redis 分布式缓存，提高了认证信息检索的效率。系统的权限验证机制不仅支持角色级别和功能级别的权限控制，还实现了多租户权限隔离以及灵活的权限组合验证策略，以适应复杂业务场景的需求。此外，系统还设计了一套统一的异常处理机制，确保从用户登录、权限校验到注销等环节的稳定性和可维护性，最后还详细介绍了利用 Playwright 实现自动化端到端测试，以及利用 Docker 和 GitHub Actions 在阿里云下实现持续集成和部署过程。

总的来说，本项目旨在开发一个基于先进的前后端分离技术架构的企业级管理系统模板，提供优质、高效的开发流程和卓越的用户体验，满足企业日益增长的复杂和定制化管理需求。

关键词：Vue3；Spring Boot；企业级管理系统；RBAC

Abstract

As the Internet technology continues to be widely adopted and information technology develops rapidly, enterprise management systems have become increasingly important in modern corporate operations. Enterprise management systems aim to improve management efficiency, promote team collaboration, optimize resource allocation, and enhance decision-making quality through information technology, thereby enhancing the company's market competitiveness. However, the current enterprise management systems still have deficiencies in terms of functionality, scalability, and user experience, and are unable to meet the growing complex and diverse management needs of enterprises.

The design goal of this project is to develop a template for an enterprise-level management system based on Vue 3 and Spring Boot, using a modern front-end and back-end technology architecture, with the aim of providing an efficient, high-quality development process and an excellent user experience, while also providing the core functionality of the management system to meet the diverse information management needs of enterprises.

To enhance the user experience of the enterprise information management system, the system supports adaptive layout and theme customization features. The system interface supports responsive design, which can automatically adjust the layout based on the client's device type and screen size. The system also supports dark and light theme customization. In terms of system development, the use of component-based and Hooks-based development patterns has improved code reusability and maintainability. TypeScript's type checking and code suggestion features have enhanced code quality and development efficiency. Additionally, the system adopts a multi-level caching strategy, including Caffeine local cache and Redis distributed cache, to improve the efficiency of authentication information retrieval. The system's permission verification mechanism not only supports role-level and function-level permission control, but also implements multi-tenant permission isolation and

flexible permission combination verification strategies to adapt to complex business scenarios. Furthermore, the system has designed a unified exception handling mechanism to ensure the stability and maintainability of the entire process, from user login, permission verification to logout. The concluding section comprehensively documents (a) the automation of end-to-end testing procedures employing Playwright's testing framework, and (b) the architectural implementation of a containerized continuous integration and delivery (CI/CD) system, orchestrated through Docker containers and GitHub Actions workflows, deployed on Alibaba Cloud's infrastructure.

In summary, this project aims to develop a template for an enterprise-level management system based on advanced technology architecture, providing an efficient, high-quality development process and an excellent user experience, to meet the growing complex and diverse management needs of enterprises.

Key Words: Vue3; Spring Boot; Enterprise management system; RBAC;

目 录

摘 要	I
Abstract	II
第一章 绪论	1
1.1 研究的背景和意义	1
1.1.1 研究背景	1
1.1.2 研究的重要性	1
1.2 企业管理系统国内外研究现状	2
1.3 研究内容与方法	2
1.3.1 研究内容	2
1.3.2 研究方法	3
第二章 项目相关技术介绍	4
2.1 整体架构选型	4
2.1.1B/S 架构核心组成	4
2.2 服务端相关技术	5
2.2.1 服务端语言选择: Java	5
2.2.2 服务端框架选择: Spring Boot 生态	5
2.2.3 控制层 Spring MVC	5
2.2.4 数据持久层 MyBatis 和 MyBatis Plus	6
2.2.5 数据库选择 Postgresql	6
2.3 客户端相关技术	7
2.3.1 前端框架选择 Vue3	7
2.3.2JavaScript 超类语言 TypeScript	8
2.3.3 层叠样式表 CSS 的预处理器和原子化框架	8
2.3.4 组件库 Element Plus	9
2.4 部署服务器	9

2.4.1 服务器	9
2.4.2 Docker	9
2.4.3 持续集成和持续部署	10
2.5 其他相关工具和技术	10
2.5.1 集成开发环境	10
2.5.1 端到端测试	10
2.6 本章小结	11
第三章 基础企业管理系统需求分析	13
3.1 企业管理系统基础需求分析	13
3.1.1 用例分析	13
3.1.2 常见的权限管理模型	14
3.1.3 基于角色的访问控制 RBAC	14
3.2 其他重要功能需求分析	15
3.2.1 多租户模型	15
3.2.2 国际化	16
3.2.3 用户界面：自适应和主题化	16
3.3 本章小结	17
第四章 基础信息管理系统设计	18
4.1 整体架构设计	18
4.2 模块功能设计	18
4.2.1 用户管理模块	19
4.2.2 个人信息管理模块	20
4.2.3 租户管理模块	20
4.2.4 租户套餐管理模块	20
4.2.5 岗位管理模块	21
4.2.6 数据字典管理模块	21
4.2.7 菜单管理模块	21

4.2.8 角色管理模块	22
4.2.9 认证管理模块	22
4.3 界面设计	23
4.3.1 设计原则	23
4.3.2 界面整体布局	24
4.3.3 登录界面	24
4.3.4 CIUD 功能页面	25
4.4 客户端设计	26
4.4.1 Element Plus 组件封装设计	26
4.4.2 自定义 hooks 封装设计	26
4.5 服务端设计	26
4.5.1 模块拆解	26
4.5.2 系统模型设计	27
4.5.3 权限认证过程	28
第五章 应用与实践	30
5.1 客户端实现	30
5.1.1 自动生成路由和布局	30
5.1.2 界面自适应实现	31
5.1.3 深色浅色主题切换	32
5.1.4 Element Plus 组件二次封装	34
5.1.5 封装 Hooks: useApi	35
5.1.6 定义接口层 fetch 接口访问	36
5.1.7 一百行实现增删查改	38
5.2 服务端实现	39
5.2.1 模块拆分实现	39
5.2.2 数据库实现	40
5.2.3 认证流程实现	44

5.2.4 功能实现	47
第六章 测试与部署	51
6.1 软件测试	51
6.1.1 功能测试	51
6.1.2 测试用例	51
6.1.3 单元测试	57
6.1.4 用 Playwright 实现端到端测试	58
6.2 运行与部署	59
6.2.1 前端运行与部署	59
6.2.2 部署环境搭建	60
6.2.3 使用 Docker、Github Actions 和阿里云实现后端 CI	62
结 论	66
参考文献	67
致 谢	68

第一章 绪论

1.1 研究的背景和意义

1.1.1 研究背景

随着互联网技术的持续普及和信息技术的迅猛发展，云计算、大数据以及人工智能的应用亦不断为传统大型企业和新兴小微企业注入新的活力。在这一时代背景下，企业管理系统在现代企业运营中的重要性日益凸显。企业管理系统致力于通过信息化手段提升管理效率、促进团队协作、优化资源配置、提高决策质量，进而增强企业的市场竞争力^[1]。

尽管如此，现行的企业管理系统在功能完备性、可扩展性以及用户体验等方面仍存在若干缺陷，难以适应企业日益增长的复杂和多样化管理需求。对于中小型企业而言，构建复杂且高质量的企业级管理后台对技术人才的要求极高。对于大型企业而言，现有的管理系统往往过于庞大，包含大量冗余功能，且由于长期缺乏技术更新和迭代，导致开发效率低下和软件性能不佳。

在当前企业信息管理系统领域，主要涵盖了 ERP（企业资源计划系统）、CRM（客户关系管理系统）、OA（办公自动化系统）、HRM（人力资源管理系统）、BI（商业智能系统）、PLM（产品生命周期管理系统）、EAM（设备资产管理系统）、代码平台和数据中台等，这些系统均基于特定的商业需求或企业内部管理流程而设计。

本项目的设计目标是开发一个基于 Vue 3 和 Spring Boot 以及相关开源技术的企业级管理系统模板^[2]，采用现代化的前后端技术架构，旨在提供高效、优质的开发流程和卓越的用户体验，同时提供管理系统的基础核心功能，以满足企业多样化的信息管理需求。

1.1.2 研究的重要性

我国对企业的信息化建设给予了高度关注，并发布了一系列政策文件，其中包括《“十四五”数字经济发展规划》和《企业信息化促进指导意见》，政策性文件中明确了信息化建设的目标和详细的实施路径。这些政策为信息化管理系统的构建提供了坚实的政策

支持。信息化管理系统可以使企业优化业务流程、提升决策效率，从而增强市场竞争力；它能够实现资源的优化配置，降低企业运营成本，从而改善企业资源配置；信息化管理系统不仅是技术工具，更是推动管理创新的关键力量。通过信息化建设，企业能够引入先进的管理经验和方法，提高整体管理水平；此外，信息化管理系统的广泛运用有助于促进传统产业的转型升级。

1.2 企业管理系统国内外研究现状

国外企业管理系统的发展起步较早，技术成熟度较高。美国、欧洲和日本等国家在企业管理系统的研究和应用方面处于领先地位，尤其是在 ERP、CRM 和 SCM 等领域积累了丰富的经验。例如，美国的 SAP、Oracle 和德国的 SAP 等企业推出的管理系统，因其功能全面、技术先进，在全球范围内得到广泛应用。然而，随着国内互联网企业的快速发展，中国在企业管理系统的研究和应用方面实现了显著突破。以华为、阿里巴巴、腾讯为代表的企业，通过自主研发的云计算、大数据和人工智能技术，推动了企业管理系统的智能化、平台化和国产化进程。特别是在多租户的 SaaS 模式^[3]、低代码平台和行业定制化解决方案方面，国内企业已经逐步赶超国际水平，特别是在基于 C/S 架构的前后端分离设计的信息化管理系统国内甚至实现了反超。形成了具有中国特色的企业管理系统生态。未来，也更好地满足了国内本地化。随着技术的不断迭代和政策的持续支持，国内外企业管理系统将在智能化、集成化和安全性方面进一步融合与创新。但国内目前很多企业内部信息管理系统存在技术老旧臃肿，用户界面以及交互体验较差的实情，整体和国际头部企业还存在一定差距。相信随着国内诸多优秀互联网企业不断发展，国内诸多企业信息化管理系统能实现对国外的追赶甚至赶超。

1.3 研究内容与方法

1.3.1 研究内容

本文聚焦于企业信息管理系统在实践中的功能瓶颈与优化路径，从开源技术生态的整合、系统设计的灵活扩展性以及国产化技术自主可控性三个维度展开研究。通过分析行业现有系统的技术痛点，结合政策导向与工程实践需求，探索如何构建一套既能满足

企业多样化管理需求、又具备长期技术演进能力的解决方案。研究以模块化架构为核心，围绕多租户数据隔离、动态权限控制、跨设备适配等关键场景展开，重点解决传统系统存在的技术僵化、维护成本高、用户体验不足等问题。在技术选型上，注重开源协议的合规性及技术替代方案的可行性，确保系统在复杂国际技术环境下的可持续性，同时通过分层设计与自动化工具链的实践，推动企业信息化建设与《“十四五”数字经济发展规划》的深度融合。

1.3.2 研究方法

本研究采用多维度交叉验证的方法，通过文献研究、技术生态分析及工程实践相结合的方式推进。一方面系统梳理国内外学术成果与行业最佳实践，对比不同技术路线的优劣势，形成适配企业需求的技术框架；另一方面基于模块化开发理念，通过分层架构设计、组件化封装与自动化测试工具的应用，构建可扩展的业务功能体系。研究过程中以实际企业管理场景为验证载体，通过功能测试与性能压测不断优化系统稳定性，同时结合政策要求与国产化趋势，在代码规范、技术替代方案等方面建立保障机制，最终形成兼具理论深度与实践价值的系统开发范式，为企业数字化转型提供可复用的方法论支持。

第二章 项目相关技术介绍

2.1 整体架构选型

为了实现基础现代企业信息管理系统的相关核心需求，本项目选择采用更适合国内开发者分工的 B/S 架构即浏览器^[4]，与此同时 B/S 架构也更适合现代企业信息管理系统的搭建、维护以及部署。

2.1.1 B/S 架构核心组成

浏览器 (Browser)：现代浏览器主要包括以 Blink 为内核的 Google Chrome、Microsoft Edge 和 以 Webkit 为内核的 Safari，主要作用是对服务器返回的 HTML、CSS 和 JavaScript 进行解析并通过一系列的复杂处理最终完成页面渲染将其呈现给用户进行交互；对应用层协议 HTTP 和 HTTPS 等相关 TCP/IP 应用层协议提供了安全高效网络支持；同时还提供了一系列客户端缓存技术的支持，如：Cookie、Storage 和 IndexedDB 等。总的来说浏览器通过对不同操作系统之间系统调用级别的各种优化与支持成为 B/S 架构的基石。

服务端 (Server)：即 B/S 架构中的 Server，主要负责接收客户端请求、处理各种复杂的业务逻辑、完成各种事务并安全可靠地存放到数据库。现代服务器通常采用分布式的服务架构，如使用像 Kubernetes+Docker 和微服务的软件开发方式来完成服务部署和编排。

网络 (Network)：通常指浏览器与服务器之间的通信，底层通常是基于 TCP/IP，应用层一般使用 HTTPS 和 HTTP，复杂场景可能需要用到 WS 和 WSS 进行 Web Socket 连接。

2.2 服务端相关技术

2.2.1 服务端语言选择：Java

现代服务端开发语言一般有 Java、Python、Go、Rust 虽然其语言核心都是基于 C 语言发展而来的，但各自分别有不同的适用的应用开发场景：

Java: 得益于 JVM 跨平台高安全性的特点，更适合企业级应用、分布式系统、金融系统和电商应用的开发。

Python: 因为其简洁的语法和强大的库支持，适合编写各种各样的脚本，在大数据人工智能以及数据分析和科学计算领域有得天独厚的优势。

Go: 与 Java 和 Python 擅长的领域不同，它得益于在高并发和部署简单，非常适合分布式的网络应用和游戏服务端。

Rust: 是近几年相当流行的一门语言，非常适合需要网络的嵌入式系统开发，由于高并发和低延迟的特点，适合开发更底层的网络应用如区块链等。

综合语言特性来看，Java 更适合开发企业级应用的服务端开发。此外由于其技术生态的庞大以及国内各大互联网以及金融行业的普及其语言地位在国内依旧是首选。

2.2.2 服务端框架选择：Spring Boot 生态

在选择 Java 作为服务端语言以后，我们可以借助其强大的生态为我们提供丰富基础公共逻辑，现代企业应用一定离不开 Spring Boot 以及其相关生态，与传统的 Spring 应用相比，Spring Boot 采用“约定大于配置”的设计理念，以及在支持 IOC (Inversion of Control) 即通过控制反转的方式注入依赖的 Spring 中 XML 配置文件配置依赖的基础上对其进一步简化，节省了大量配置，极大增强了项目的可维护性。不仅如此 Spring Boot 还提供了大量实用稳定的 Starter 进一步简化了 Spring 应用的配置。

2.2.3 控制层 Spring MVC

Spring MVC 是 Spring 框架中的首当其冲模块,用于构建基于 MVC 设计模式的 Web

应用程序。它为开发者提供了一种高效的方式来开发 Web 层应用。Spring MVC 的核心思想是将请求处理、业务逻辑和视图渲染分离，从而提高代码的可维护性和可扩展性。框架核心是围绕 DispatcherServlet 负责将客户端的请求通过 HandlerMapping 找到对应的 Controller 完成业务后返回 ModelAndView 给用户对应的服务。

2.2.4 数据持久层 MyBatis 和 MyBatis Plus

MyBatis 是一款轻量级且功能强大的持久层框架，旨在简化数据库操作并提高开发效率。它通过 XML 自定义 SQL 或注解将 Java POJO 与 SQL 语句关联映射，让开发者能够直接编写灵活的 SQL，同时避免了传统 JDBC 的繁琐操作。MyBatis 支持动态 SQL、存储过程调用以及复杂的结果集映射，适用于需要精细控制 SQL 的场景。其核心组件包括 SqlSessionFactory、SqlSession 和 Mapper 接口，通过配置文件和接口定义，MyBatis 能够自动生成数据库操作的实现类。与 Spring 框架的无缝集成进一步增强了其易用性，支持事务管理和依赖注入。MyBatis 的优势在于其灵活性、高性能和易于调试的特点，特别适合处理复杂的查询和大规模数据操作。在企业级信息管理系统中，MyBatis 能够有效降低数据库访问的复杂度，提升系统的可维护性和扩展性，是现代 Spring Boot 应用中不可或缺的持久层解决方案。

2.2.5 数据库选择 Postgresql

现代数据库主要分为两类，即关系型数据库（RDBMS）以及非关系型数据库（NoSQL）。其中关系型在企业信息管理系统通常作为主数据库。市面上关系型数据库主要有 Postgresql^[5]、MySQL 和 Oracle。数据库的技术选型需要综合考虑性能、功能、成本和适用场景等因素。MySQL 和 Oracle 是两种常见的关系型数据库，各有优劣：MySQL 以其轻量级、开源免费和易于部署等特点，非常适合中小型应用和快速开发场景；而 Oracle 则以其强大的功能集、高可用性和高级安全性著称，适合大型企业级应用，但其高昂的成本和复杂的维护要求可能不适合预算有限的项目。同时 MySQL 近年来由于其设计之初考虑不足的问题慢慢凸显，许多问题多年以来一直未得到解决，如：limit 查询性能问题，数据存储内置 bug 等，再者因为被 Oracle 收购后存在版权风险日益背离开源初衷用户信任度直线下降。

相比之下，PostgreSQL 底层严格遵守 ACID（原子性、一致性、隔离性和持久性）确保事务可靠的同时还结合了两者的优势：它提供了与 Oracle 类似的高级功能（如复杂查询优化、JSON 支持和事务处理）；保持了开源免费的特性，适合从中小型到大型的各种应用场景。此外，PostgreSQL 的活跃社区和丰富的扩展插件使其在灵活性和可扩展性上表现优异。因此，综合考虑功能、成本和适用性，我们选择 PostgreSQL 作为开发管理系统的首选数据库，以满足系统的高性能、高可靠性和未来扩展需求。

2.3 客户端相关技术

2.3.1 前端框架选择 Vue3

随着客户端的运算能力的逐步提升和移动互联网的深度普及。SPA（Single-Page-Application）通过动态加载内容而不是重新刷新整个页面来提供流畅的用户体验，已成为 Web2.0 后时代的不二选择。前端领域目前流行的框架是一个“三足鼎立”的时代，包括 Vue、React 和 Angular。三者中 Angular 最早出现由 Google 孵化并维护，提供了完整的 MVC 框架和强大的 Typescript 支持，因此非常适合后端开发人员上手。React 由原 Facebook 公司核心成员创建并维护（现：Meta 公司），因其单向数据流的和 JSX 语法作为一个轻量的 JavaScript 库而深受广大开发者的喜爱和需要高度定制化企业的青睐。

Vue 和 Angular 同出师门，和 React 不同的是 Vue 是一个完整的 MVVM（Model-view-viewmodel）框架，其渐进式设计允许开发者根据项目需求逐步引入功能，既适合小型项目快速上手，同样也适合大型企业级应用开发。由于其作者尤雨溪是中国人，因此早期凭借其优秀的中文文档支持，所以国内早期市场占有率颇高，随着后期 Vue 相关生态不断地更新迭代和壮大，例如 Vuex 状态管理、VueRouter 路由管理以及 Element UI 等。得益于 template 模板和许多其他优雅的设计，Vue2 在前端领域有自己的一席之地。随后的 Vue3 版本更是颠覆性地使用当时并不完全成熟的 Typescript 进行重新开发，虽然早期升级和发展受到了一定挑战，但随后 Vite 构建工具推出，得到了市场强烈的反馈。如 Piniajs 状态管理、VueRouter4 路由管理以及 Element Plus 也相继推出。截至 2025 年已经成为三者中性能最强的前端框架。

因此选择 Vue3 来开发现代企业信息管理系统，不仅可以借助其完整的框架生态创

建高级的前端系统，而且还可以借助其更好地中文开发社区减少初级开发人员的学习成本。

2.3.2 JavaScript 超类语言 TypeScript

TypeScript 是 JavaScript 的超集，是微软内部团队开发并维护，目的是在为 JavaScript 提供可选的静态类型检查和面向对象编程特性。它极大地扩展了 JavaScript 的语法，支持 Class 类、Interface 接口、Generics 泛型、Enum 枚举等其他高级特性，同时能够完全地兼容现有的 JavaScript 代码。TypeScript 通过编译时类型检查，能够有效减少运行时错误，提升代码的可维护性和可读性，特别适合大型项目和企业级项目的开发。此外，TypeScript 提供了强大的工具支持（如自动补全、代码重构等），进一步提高了开发体验和开发效率，TypeScript 在 Angular、Vue 3 等框架中得到了广泛应用，成为构建健壮、可扩展应用的重要工具。

2.3.3 层叠样式表 CSS 的预处理器和原子化框架

CSS (Cascading Style Sheets) 即层叠样式表是一种用于描述 HTML 文档外观的样式语言，通过选择器，如 ID 选择器、元素选择器和类选择器等，将样式规则应用到特定的元素上，从而控制页面的布局、颜色、字体等视觉效果。

Sass (Syntactically Awesome Style Sheets) 是一种 CSS 预处理器，嵌套、混合 (Mixins)、提供了变量 (Var) 和继承等高级功能，例如颜色处理等的内置函数。使得编写和维护样式表更加高效和灵活。Sass 支持两种语法格式：一种是缩进式的 sass 语法，另一种是类似 CSS 的 scss 语法。通过编译，Sass 可以将这些高级特性转换为标准的 CSS，帮助开发者更好地组织、协作和管理样式代码，特别适合大型项目和企业级项目的开发。

UnoCSS 是一款超酷的原子化 CSS 框架，它通过按需生成实用类 (Utility Classes) 来打造样式，不仅大幅压缩了 CSS 文件体积，还带来了超高的灵活性和性能优化。它像个观察者一样，动态扫描代码中的类名，只生成真正用到的样式，完美避开了传统 CSS 框架中那些冗余的样式代码，特别适合现代 Web 开发中对极致性能和轻量化的追求。UnoCSS 的简洁设计和高效表现，让它成为团队协作、构建轻快、高性能应用的绝佳拍档!

2.3.4 组件库 Element Plus

Element Plus 是一套基于 Vue 3 的桌面端组件库，提供了丰富的高质量 UI 组件（如表格、表单、对话框、导航等），并支持主题定制和国际化，极大地提升了开发效率。在企业信息管理系统中，Element Plus 的组件库能够快速搭建出美观、一致的用户界面，其强大的表格和表单组件特别适合处理复杂的数据展示和交互需求。同时，Element Plus 的高度可定制性和良好的性能优化，使得开发者能够轻松实现系统的功能扩展和用户体验优化，成为构建现代化、高效企业信息管理系统的得力工具。

2.4 部署服务器

2.4.1 服务器

在互联网早期企业一般会选择搭建自己的物理服务器，但随着云计算、云存储以及以亚马逊为首的云服务提供商的崛起，弹性计算服务 (Elastic Compute Service, 简称 ECS) 逐渐成为企业级应用部署的主流选择。ECS 作为一种可伸缩的计算服务，让企业不再需要前期投入大量资金购置硬件设备，而是可以根据实际需求灵活调整计算资源。用户可以通过 Web 服务页面轻松管理计算资源，实现服务器的快速部署和弹性伸缩。相比传统物理服务器，ECS 具有按需付费、快速部署、高可用性、安全可靠等优势，特别适合中小企业的业务部署和开发测试环境搭建。云服务器还提供了完善的监控和运维工具，帮助企业更好地管理和优化其 IT 资源。

2.4.2 Docker

在现代软件开发和部署领域，特别是企业级开发，Docker^[6]容器技术的出现标志着一个重要的技术变革。通过将应用及其运行环境、依赖库、配置文件等打包成标准化的容器单元，Docker 有效解决了“在我的机器上可以运行”的环境一致性问题。其采用分层文件系统和镜像复用机制，显著减少了存储和传输开销。这种容器化技术不仅简化了应用的部署流程，还为微服务架构和 DevOps 实践提供了强有力的技术支撑。在当前企业级开发中，Docker 已成为标准化交付和持续集成/持续部署 (CI/CD) 流程中不可或缺的基础设施。

2.4.3 持续集成和持续部署

在当代软件工程实践中，持续集成（CI）和持续部署（CD）已成为确保软件质量和快速交付的关键流程。CI/CD 通过自动化构建、测试和部署，显著降低了人为错误，提高了开发团队的协作效率。本项目选择 GitHub Actions 作为 CI/CD 工具，结合 Docker 容器技术和阿里云容器镜像服务，构建了一套完整的自动化部署方案。

这套方案的优势在于：首先，GitHub Actions 提供了丰富的预置动作和完善的触发机制，能够精确控制构建流程；其次，Docker 的标准化容器保证了应用在不同环境中的一致性；最后，阿里云私有容器镜像服务不仅提供了安全可靠的镜像存储，还能与云服务器实现无缝集成。当开发人员提交代码到主分支时，系统会自动触发构建流程，执行单元测试，构建 Docker 镜像并推送到阿里云私有仓库，最终完成自动部署。这种自动化的工作流极大地提升了开发效率，降低了运维成本，使团队能够专注于业务功能的开发和创新。

此外，通过合理配置缓存策略和并行任务，可以进一步优化构建速度，提高资源利用效率。在实践中，该方案已经证明能够有效支撑企业级应用的快速迭代和持续交付需求。

2.5 其他相关工具和技术

2.5.1 集成开发环境

工欲善其事必先利其器，在企业开发过程中，集成开发环境 IDE 能在代码提示、效率提升以及团队协作起到重要作用。前端我们选用 Visual Studio Code 作为首选开发工具并配合 Vue Official、ESLint、Prettier、UnoCss 以及内置 Typescript 插件使用。后端则使用 Java 开发的标杆 IDE：IntelliJ IDEA。

2.5.1 端到端测试

端到端测试（End-to-End Testing，简称 E2E 测试）就像是给整个软件系统做一次“全身体检”。它从用户的角度出发，模拟真实场景下的完整操作流程，验证整个系统是

否像设计的那样正常运行。在 Vue 应用的端到端测试领域, Cypress 和 Playwright 是两个最主流的测试工具。

Cypress 作为早期的主流测试工具, 提供了优秀的开发体验和直观的测试运行器界面。其实时重载、时间旅行调试和自动等待机制等特性, 使得测试编写和调试变得简单。然而, Cypress 存在一些限制, 如不支持多标签页测试、跨域限制等。

相比之下, Microsoft 开发的 Playwright 提供了更全面的功能支持: 支持所有主流浏览器 (Chromium、Firefox、WebKit)、强大的自动等待机制, 减少显式等待代码、内置移动设备模拟, 支持响应式测试、支持多标签页和 iframe 测试、优秀的并行测试执行能力以及完善的调试工具和追踪功能。

考虑到企业管理信息管理系统项目的长期发展和测试场景的完整性, 选择 Playwright 作为端到端测试工具更为合适。它不仅能满足当前的测试需求, 还为未来的测试扩展提供了更大的可能性。同时, 其完善的文档和活跃的社区也为开发团队提供了良好的支持。

2.6 本章小结

本章详细分析了现代企业信息管理系统的技术选型, 从整体架构到具体技术栈的选择, 为系统的设计与实现奠定了坚实基础。首先, 在整体架构上, 选择了适合国内开发者分工的 B/S 架构, 其核心由浏览器、服务端和网络组成, 能够满足系统的搭建、维护和部署需求。在服务端技术选型中, Java 凭借其跨平台、高安全性和强大的生态支持成为首选语言, 结合 Spring Boot 生态简化了开发流程, 并通过 Spring MVC 实现了高效的请求处理和业务逻辑分离。数据持久层采用 MyBatis 和 MyBatis Plus, 提供了灵活的 SQL 映射和高性能的数据库操作能力。数据库方面, 经过对 MySQL 和 Oracle 的对比分析, 最终选择了兼具功能强大和开源优势的 PostgreSQL, 以满足系统的高性能和高可靠性需求。

在客户端技术选型中, 前端框架选择了 Vue 3, 其渐进式设计和强大的生态支持 (如 Vuex、Vue Router 和 Element Plus) 能够快速构建高效、可维护的用户界面。TypeScript 作为 JavaScript 的超集, 提供了静态类型检查和面向对象编程特性, 显著提升了代码的可读性和可维护性。样式表方面, 结合 Sass 的预处理器功能和 UnoCSS 的原子化框架, 实现了样式代码的高效管理和性能优化。最后, Element Plus 组件库为系统提供了丰富

的 UI 组件，进一步提升了开发效率和用户体验。

综上所述，本章的技术选型充分考虑了企业信息管理系统的功能需求、开发效率和未来扩展性，为后续的系统设计与实现提供了可靠的技术支持。

第三章 基础企业管理系统需求分析

3.1 企业管理系统基础需求分析

随着企业规模的扩张及业务流程的日益复杂化，传统的手工管理方式已无法适应现代企业的发展需求。国内众多企业在信息管理系统方面面临技术陈旧、系统臃肿的问题，同时，对于小微型企业而言，构建企业信息管理系统所需的成本较高。为了解决这些问题，提升管理效率，提供先进的技术解决方案，并为从无到有的企业信息化建设提供启动模板，本研究项目致力于开发一套现代化的企业信息管理系统模板。该系统模板将集成用户管理、租户管理、部门管理、岗位管理、菜单管理、角色管理、国际化支持以及字典管理等多项功能模块。此外，系统还支持内容管理、财务核算、供应链管理等定制化业务需求的扩展，以使用户能够基于此模板方便地进行业务拓展。

3.1.1 用例分析

通用的信息管理系统一般角色分为普通用户，管理员，和超级管理员。超级管理员主要负责分配租户即创建某个租户的管理员。特定租户的管理员则可以对本企业的组织架构创建用户、权限分配和授权工作。普通用户则根据自身功能使用岗位和部门下的权限使用已分配好的功能。具体用例可参考（如图 3-2-1）

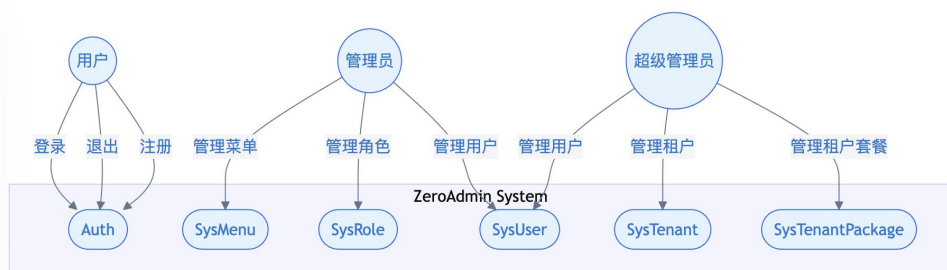


图 3-1-1

3.1.2 常见的权限管理模型

在企业信息化管理系统中核心的功能是要确保不同用户能获取和操作对应岗位职责的任务，同时也能防止机密数据越权泄露的情况发生。因此系统核心的功能是权限管理的控制，权限管理控制模型主要有三种：访问控制列表 ACL(Access Control List)，基于属性的访问控制 ABAC (Attribute-Based Access Control) 和 RBAC (Role-Based Access Control) 基于角色的访问控制^[7]（如图 3-2-2）。

ACL 访问控制列表，是三者中最简单的一种模型。核心原理是基于对象和主题之间的关系来控制资源访问。适合在简单的用户分享资源、简单网盘的业务逻辑中使用。

ABAC 基于属性的访问控制，又称为基于策略的访问控制 PBAC (Policy-Based Access Control) 或基于声明的访问控制 CBAC(Claims-Based Access Control)。其特点是适合需要极为灵活策略来决定用户是否有特定资源权限的场景，通常用于 IoT 物联网智能设备、政务和军事系统、云计算和云服务（如：阿里云 RAM）等。因为其过于复杂，成本过高不适用于开发通用的基础企业内部管理系统，因而更适合需要同时对内和对外的综合型系统。

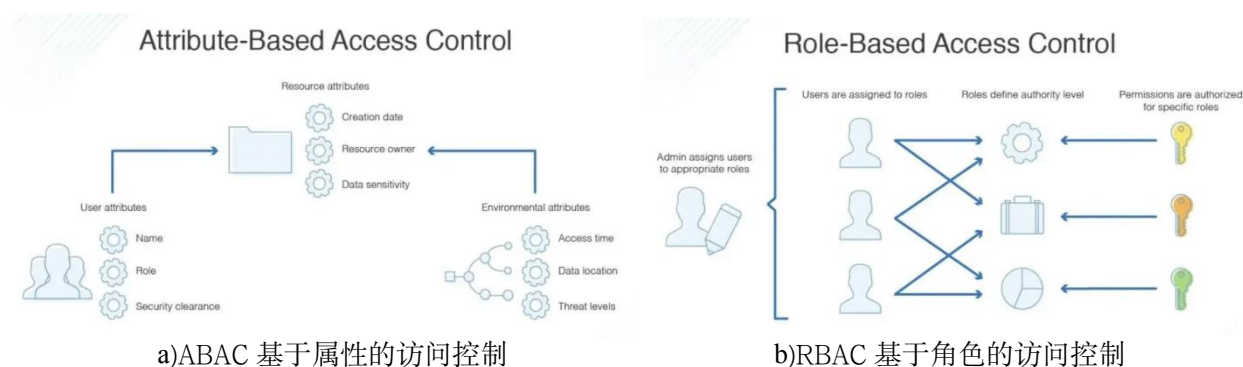


图 3-1-2

3.1.3 基于角色的访问控制 RBAC

RBAC 基于角色的访问控制，此模式是前两者折中的一个方案，能够满足企业绝大部分场景的同时配置方式也非常适合企业常见的管理方式，为了实现 RBAC 权限管理模式，通常需要用户管理、认证管理和角色管理几个模块相互配合并添加部门管理、岗位管理和菜单管理加以辅助来实现一个企业级别的基于角色的访问控制。

总的来说，基于角色的访问控制（RBAC）提供一种访问控制方法，通过将权限和角色关联或策略绑定，并将角色通过分配给指定用户来管理用户对系统资源的访问。在 RBAC 模型中，角色是权限的集合，用户通过其分配的角色获得相应的权限。RBAC 的需求分析主要包括以下几个方面：

1.角色定义：系统需要明确定义不同的角色，每个角色代表一组特定的权限。例如，系统管理员、普通用户、访客等角色。每个角色的权限应根据业务需求进行详细定义，以确保角色的权限范围清晰明确。

2.权限管理：权限是系统中可以执行的操作或访问的资源。系统需要提供权限管理功能，允许管理员创建、修改和删除权限。权限应与具体的操作或资源绑定，例如查看、编辑、删除数据等。

3.角色与权限的关联：系统需要支持将权限分配给角色。每个角色可以拥有多个权限，一个权限也可以分配给多个角色。这种多对多的关系需要在系统中进行有效管理，以确保权限分配的灵活性和准确性。

4.用户与角色的关联：系统需要支持将角色分配给用户。每个用户可以拥有多个角色，一个角色也可以分配给多个用户。通过角色的分配，用户可以继承角色所拥有的权限，从而实现对系统资源的访问控制。

5.访问控制检查：系统在用户准备访问所需资源或执行相关特权操作时，系统需要进行权限检查。根据用户的角色和角色所拥有的权限，决定是否允许用户执行相应的操作。这种检查应在系统的各个层面进行，以确保安全性。

3.2 其他重要功能需求分析

3.2.1 多租户模型

在现代企业信息管理系统中，出现许多远端部署的 SaaS（Software-as-a-Service），它是一种搭建在云计算平台之上的软件服务模式，用户可以通过全球或部分区域互联网进行操作的平台。除此之外，目前国内许多企业呈现集团化、业务线分离和事业部门等方式对业务进行拆分，多数内部管理系统也简单根据业务进行划分，虽然能保证业务基本正常运行，但是由于系统级别的分割，使得很难对同一个企业内各个系统进行统筹配

置；而且存在许多重复性开发造成了成本的浪费。

多租户模型是 SaaS 系统中重要的一个概念，它是一种能够支持多个租户（如同一企业的不同业务线或部门），共享同一套系统实例的架构设计模式，目的是在实现资源的高效利用和成本的优化，同时确保各个租户之间的多种数据和不同配置完全隔离。以下是多租户模型的核心需求描述：

租户管理：系统需支持租户的基本信息管理，并提供租户状态的启用与禁用功能，同时记录状态变更日志。

数据隔离：确保不同租户的数据完全隔离，租户只能访问和操作自己的数据，支持基于租户 ID 的数据过滤和权限控制。

配置管理：支持为每个租户配置独立的菜单、权限模板和系统参数，满足租户的个性化需求等。

资源分配：为每个租户分配独立的资源（如存储空间、计算资源），并监控资源使用情况，支持资源配额管理以防止资源滥用。

3.2.2 国际化

国际化 i18n (Internationalization) 是企业信息管理系统的重要功能之一，目的是在支持多语言、多地区和多文化的用户使用系统。随着改革开放几十年的不断发展，虽然全球化进程在当下日益困难，但对外开放是符合多极化世界的基本路线，国内企业出海成功案例也不胜枚举，得益于国内互联网许多优秀的实践，国内信息化产品在国际上有一定领先地位，需要跨国合作的企业也日益增多，因此实现国际化的需求也成了许多软件必不可少的一部分。

本系统支持最基础的多语言，即语言切换：系统应支持用户根据个人偏好切换界面语言，考虑到成本和 AI 人工智能发展迅速的当下，仅实现象形文字（中文）和表音文字（英语）即可满足绝大部分应用场景。

3.2.3 用户界面：自适应和主题化

为了提升企业信息管理系统的用户体验，用户界面需要支持自适应布局和主题个性化功能，以满足不同设备和用户偏好的需求。系统界面需要支持响应式设计，可以根据

用户的客户端设备类型（如 PC、平板、手机）以及屏幕尺寸自动调整布局，确保交互界面在不同设备上均能提供良好的显示效果和操作体验。在企业多场景使用是非常有必要的，例如：现场销售人员一般需要平板系统对接系统进行进销存相关业务处理。同样在用户界面设计中主题化也成了标准化设计中不可或缺的一个功能，其中最基础的是深色和浅色主题。

3.3 本章小结

本章围绕现代企业信息管理系统的需求展开，从基础需求到功能性能需求，再到其他重要功能需求，层层深入，为系统的设计与实现提供了清晰的蓝图。

首先，在基础需求分析中，我们明确了系统开发的初衷：传统管理方式效率低、技术老旧，而小微企业构建系统成本又高，因此我们需要打造一套现代化的企业信息管理系统模板，帮助企业从零到一快速搭建高效的管理平台。这套系统不仅涵盖了用户管理、租户管理、部门管理、岗位管理、菜单管理、角色管理、国际化和字典管理等核心功能，还支持用户在此基础上灵活扩展，满足更多定制化需求。

接着，在功能性能需求分析中，我们重点探讨了权限管理的三种模型：ACL、ABAC 和 RBAC。经过对比，RBAC 凭借其灵活性和高效性脱颖而出，成为企业信息管理系统的首选。RBAC 通过将权限与角色绑定，再将角色分配给用户，实现了权限的精细化管理，既保证了数据安全，又简化了权限配置流程，完美契合企业的实际需求。

在其他重要功能需求分析中，我们深入探讨了多租户模型、国际化和用户界面设计等关键功能。多租户模型让多个租户可以共享同一套系统实例，既节省了资源，又确保了数据隔离，特别适合集团化企业或 SaaS 平台。国际化功能则通过多语言支持和本地化格式，让系统能够轻松应对跨国业务需求，助力企业走向全球。而用户界面的自适应和主题化设计，则让系统在不同设备上都能展现出最佳效果，同时支持用户根据自己的喜好切换主题，真正做到了“千人千面”。

本章的需求分析不仅明确了系统的核心功能和技术方向，还为后续的设计与开发提供了扎实的理论基础。通过这些功能的实现，我们的企业信息管理系统将能够帮助企业提升管理效率、降低运营成本，同时为用户带来更智能、更友好的使用体验。接下来，就让我们把这些需求转化为实际的功能，开始系统的设计与开发。

第四章 基础信息管理系统设计

4.1 整体架构设计

本系统整体架构基于 B/S 模式，客户端采用 Vue3 技术栈进行开发；服务器端则基于 Spring Boot 框架构建；客户端与服务器端通过 HTTP 协议，利用 Restful API 进行数据交互；数据库系统选用 PostgreSQL 作为主要数据存储，同时采用 Redis 作为数据缓存层。服务端应用部署于 Undertow 容器中，客户端与 Restful API 的访问则通过 Nginx 进行反向代理（参见图 4-1-1）。

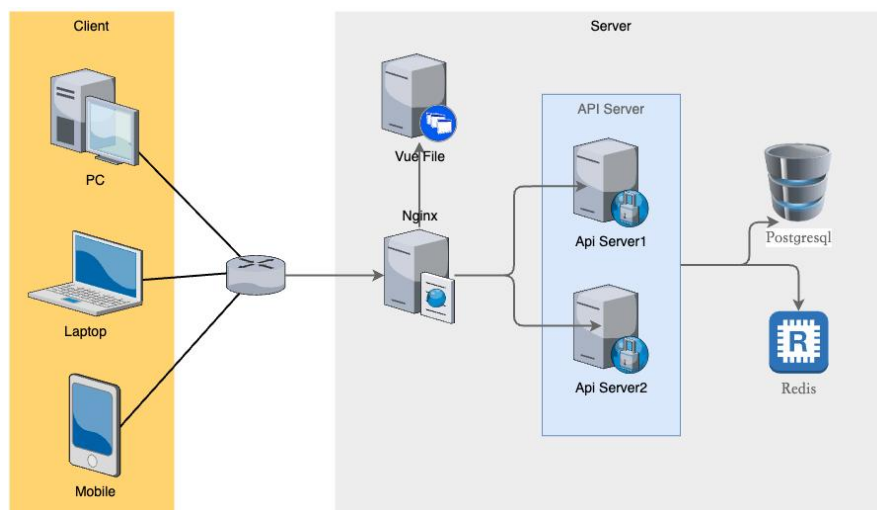


图 4-1-1 整体架构示意图

4.2 模块功能设计

本企业信息管理系统采用模块化设计架构，主要包含七大核心功能模块：租户模块（实现多租户隔离与资源分配）、用户模块（负责用户账号管理与认证）、岗位模块（定义组织职位体系）、部门模块（构建企业组织架构）、菜单模块（管理系统功能权限树）、角色模块（配置权限集合）以及字典模块（维护系统基础数据字典）。系统基于 RBAC（基于角色的访问控制）模型，将角色成员划分为三个层级：普通企业用户（具有基础业务操作权限）、租户管理员（拥有租户内系统管理权限）和超级管理员（具备全系统

最高管理权限)，不同角色通过权限矩阵实现细粒度的功能管控。如（图 4-2）所示，这三类角色形成金字塔式权限结构，下级角色权限始终为上级角色权限的真子集，确保系统权限管理的安全性可扩展性。各模块间通过标准化接口进行数据交互，共同构成完整的企业级信息管理解决方案。

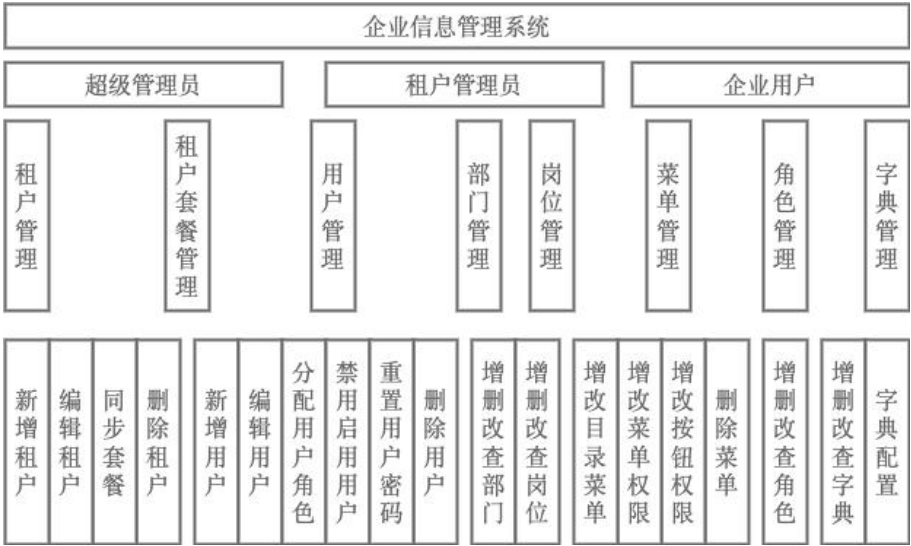


图 4-2-1 层次模块结构图

4.2.1 用户管理模块

- 用户列表检索：实现对所有用户信息的检索功能。
- 用户列表导出：支持用户信息导出至 Excel。
- 用户信息查询：实现对当前登录用户信息的查询功能。
- 用户详细信息查询：依据用户 ID，检索并展示用户详细信息。
- 用户信息新增：提供添加新用户信息的接口。
- 用户信息编辑：实现对现有用户信息的编辑功能。
- 用户信息删除：根据 userId 执行用户信息的删除操作。
- 用户密码重置：提供用户密码重置功能。
- 用户状态变更：实现用户状态启用\禁用的状态修改功能。
- 用户授权角色查询：依据 userId，检索用户所授权的角色信息。
- 用户角色授权：实现为用户分配角色的授权功能。

部门树列表获取：提供获取部门树列表的功能。

部门用户信息检索：实现对指定部门下所有用户信息的检索功能。

4.2.2 个人信息管理模块

用户信息获取：实现当前登录用户个人信息的检索与展示。

用户信息编辑：修改当前登录用户的个人信息。

密码重置功能：允许当前登录用户密码的重置操作。

4.2.3 租户管理模块

查询租户列表：实现对所有租户信息的检索与展示。

导出租户列表：支持将租户信息导出为 Excel 格式的文件。

获取租户详细信息：允许通过租户 ID 查询并获取租户的详细资料。

新增租户：提供添加新租户的接口。

修改租户信息：实现对现有租户资料的更新与编辑。

状态变更：允许对租户的激活状态进行调整。

删除租户：提供通过租户 ID 进行租户信息的移除操作。

动态切换租户功能：实现对当前登录用户所属租户的即时切换。

清除动态租户功能：支持清除当前登录用户的动态租户设置。

同步租户套餐信息功能：确保租户的套餐信息保持最新状态。

同步租户字典信息功能：保证租户字典信息的同步更新。

4.2.4 租户套餐管理模块

检索租户套餐清单：获取全部租户套餐清单。

检索租户套餐下拉菜单选项清单：获取租户套餐的下拉菜单选项清单。

导出租户套餐清单：将租户套餐清单导出为 Excel 格式文件。

获取租户套餐详细资料：依据租户套餐 ID 获取其详细资料。

新增租户套餐：添加新的租户套餐项。

修改租户套餐：更新现有租户套餐信息。

状态变更：调整租户套餐的状态。

移除租户套餐：依据租户套餐 ID 移除特定套餐项。

4.2.5 岗位管理模块

获取岗位列表：获取所有岗位的列表。

导出岗位列表：将岗位列表导出为 Excel 文件。

根据岗位编号获取详细信息：根据岗位 ID 获取岗位的详细信息。

新增岗位：添加新的岗位。

修改岗位：修改现有的岗位。

删除岗位：根据岗位 ID 删除岗位。

获取岗位选择框列表：获取岗位的选择框列表。

4.2.6 数据字典管理模块

查询字典类型列表：获取所有字典类型的列表。

导出字典类型列表：将字典类型列表导出为 Excel 文件。

查询字典类型详细：根据字典 ID 获取字典类型的详细信息。

新增字典类型：添加新的字典类型。

修改字典类型：修改现有的字典类型。

删除字典类型：根据字典 ID 删除字典类型。

刷新字典缓存：刷新字典缓存数据。

获取字典选择框列表：获取字典类型的选择框列表。

4.2.7 菜单管理模块

获取用户路由信息：查询当前用户可访问的系统路由列表。

查询完整菜单列表：获取系统中所有菜单项的完整清单。

查看菜单详情：通过菜单 ID 查询指定菜单的详细信息。

获取菜单树形下拉数据：以树形结构返回菜单数据，方便下拉选择。

查询角色关联菜单树：根据角色 ID 获取该角色配置的菜单权限树。

查询租户套餐菜单树：按租户套餐 ID 获取套餐包含的菜单权限结构。

创建新菜单：在系统中新增一个菜单项。

编辑菜单信息：修改现有菜单的配置内容。

删除菜单项：根据菜单 ID 移除指定的菜单。

4.2.8 角色管理模块

查询角色列表：获取系统中所有角色的完整清单

导出角色数据：将角色列表导出为 Excel 格式文件

查看角色详情：通过角色 ID 查询特定角色的详细信息

新增角色：在系统中创建新的角色

编辑角色信息：更新现有角色的基本信息

调整数据权限：修改角色对应的数据访问权限

变更角色状态：启用或禁用指定角色

删除角色：根据角色 ID 移除指定角色

获取角色选择项：获取用于下拉选择的角色选项列表

查询已授权用户：查看已分配该角色的用户名单

查询可授权用户：查看尚未分配该角色的用户名单

撤销用户授权：取消单个用户的角色授权

批量撤销授权：一次性取消多个用户的角色授权

批量分配角色：一次性为多个用户授予角色

查询角色部门树：获取该角色关联的部门树形结构

4.2.9 认证管理模块

登录方法：用户登录。

退出登录：用户退出登录。

用户注册：用户注册。

登录页面的租户下拉框：获取登录页面的租户下拉框列。

验证码管理：生成并返回验证码信息。

4.3 界面设计

4.3.1 设计原则

在企业信息管理系统界面设计中，需要综合考虑功能性、技术实现、交互设计、用户体验和响应式等多个方面。

在进行系统设计时，功能性是至关重要的一个方面，它要求我们从模块化的角度出发，深入思考如何构建一个高效且易于维护的系统。首先，我们需要根据不同的功能模块来抽离出它们的公共特性。以企业信息管理系统为例，其中的增删查改 CRUD

(Create-Read-Update-Delete) 操作是核心功能，它们可以被抽象出来，形成通用型的模块化组件。通过这样的设计，我们不仅能够提高代码的复用率，还能够使得系统的各个部分更加独立，便于后续的升级和维护。这种模块化组件化的设计方法，不仅适用于企业信息管理系统，也适用于其他需要处理复杂数据和业务逻辑的系统。

在技术实现中，需考虑 Web 前端样式的复杂度和浏览器兼容性。这涉及对主流和非主流浏览器的测试，确保网页在不同环境下用户体验一致。同时，关注性能优化，如减少页面加载时间，提高交互响应速度，保证代码的可维护性和可扩展性。这些因素影响网站成功，需在设计和开发阶段考虑。

在进行交互设计时，本系统主要需要考量以下方面：在表单设计中，需选择适当的输入组件以适应不同字段；在操作便捷性方面，应确定常用操作的入口，确保最频繁使用的功能得以显著展示，并支持必要的批量操作；应尽量采用弹窗替代页面跳转，并在执行删除等潜在风险操作前，弹出二次确认框以避免误操作。

在设计响应式系统时，应确保内容能够多种尺寸，包括个人电脑、平板以及手机等，实现恰当的展示。对于现代企业信息管理系统而言，特别需要确保其在个人电脑与平板电脑上的展示效果及操作性。对于那些结构复杂或界面设计精细的系统，必须提供适应不同屏幕尺寸或特定使用场景的自适应设计方案。

4.3.2 界面整体布局

企业信息管理平台的整体布局设计通常遵循一定的标准模式，这种模式相对比较固定，一般都采用左边固定宽度的菜单栏、顶部固定的吸顶功能区，以及不同功能的页面区域所构成。这种布局方式在视觉上为用户提供了清晰的导航路径，同时也保证了操作的便捷性。用户在使用过程中可以快速地通过左侧的菜单栏找到所需的功能模块，而顶部的吸顶功能区则提供了快速访问常用功能的途径。页面的主体区域则根据不同的功能需求展示相应的操作界面，使得用户能够专注于当前的任务。这种布局设计不仅提高了工作效率，还增强了用户体验，使得整个平台的使用变得更加直观和高效。(如图 4-5-2)。

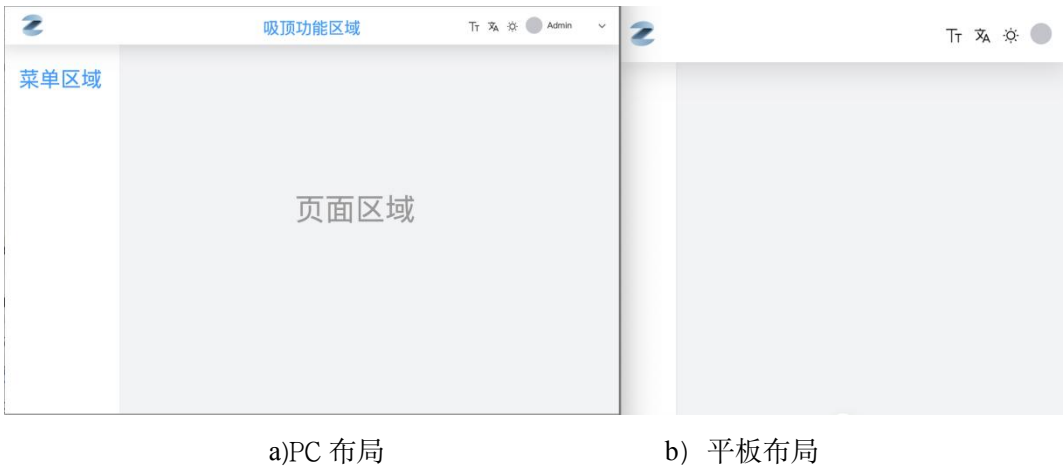


图 4-5-2 布局示意图

4.3.3 登录界面

登录界面由左右两个部分组成，左侧部分为登录侧边栏，右侧部分为登录表单区域。侧边栏采用蓝色背景，呈现欢迎信息，并可根据需求添加品牌宣传内容或广告位，从而提升视觉吸引力。右侧的登录表单区域则以居中方式展示，整体布局简洁明了，便于用户迅速定位登录入口。此外，该界面还实现了对移动设备和平板电脑的适配(如图 4-5-3)，确保在移动和平板显示模式下，登录表单整体居中展示保持功能的完整性。

登录表单采用垂直布局，依次排列的表单元素包括：租户选择下拉菜单、用户账号名称输入框、登录密码输入框、验证码及其输入框、登录状态保持选项复选框以及登录

按钮。



图 4-5-3 登录界面

4.3.4CIUD 功能页面

信息管理系统中，增删查改功能是不可或缺的核心页面，同时也是使用频率最高的功能区域。因此在设计上，必须突出其主要功能。（如图 4-5-4）包含搜索区域以及搜索功能区、二维表格展示区域以及表格行操作区域、页面底部分页导航器和侧边筛选器。



图 4-5-4 CIUD 功能页面

4.4 客户端设计

4.4.1 Element Plus 组件封装设计

为了满足上一节中的前端设计的同时我们需要对 Element Plus 组件进行二次封装，进行二次封装的必要性主要有两个方面。一方面在企业开发流程中一般都需要准确地还原设计稿；另一方面组件库本身是为了通用性需求出发许多组件本身不能很方便地实现具体的业务或需要书写大量冗余逻辑和代码。因此为了精准还原 UI 的同时，又能提高代码质量降低维护成本的，进行二次封装是非常有必要的。

对于企业级信息管理系统最常见的增删查改功能进行封装，查询对应列表查询我们可以对 el-table 组件进行二次封装；新增和修改的功能我们可以用表单组件 (el-form) 和弹窗组件 (el-dialog) 以及众多表单项组件 (如 el-input、el-number、el-select、el-datetime) 等进行组合式的封装。

4.4.2 自定义 hooks 封装设计

在 Vue3 框架中，采用 Composition API 的开发范式，开发者能够通过 hooks 技术封装响应式功能，显著减少响应式编程的代码规模，并有效提升代码及系统的质量。针对网络接口调用，可以实现一个名为 useApi 的 hooks，进一步地，实现一个名为 useTable 的 hooks 以处理查询分页数据；对于常规的表单提交操作，可以构建一个名为 useModal 的 hooks 以及一个用于动态配置表单的 useForm 的 hooks。这些功能的实现细节较为抽象，建议参阅后续章节中具体案例的详细阐述。

4.5 服务端设计

4.5.1 模块拆解

在进行业务拆分的过程中，我们选择了使用 Maven Modules 来实现模块层面的拆分。这种拆分方式允许我们将整个系统架构划分为三个主要部分：首先是管理后台的入口模块 (zero-admin)，其次是基础模块组件 (zero-base)，最后是业务模块 (zero-modules)。在基础模块中，我们包含了核心模块 (zero-base-core)，它涵盖了 Spring 框架的核心功

能、验证注解、面向切面编程（AOP）等基础性功能。除此之外，我们还设计了 Web 服务相关功能模块（zero-base-web），用于处理与 Web 服务相关的各种需求。认证服务模块（zero-base-satoken）则负责提供安全认证机制，确保系统的安全性。多租户模块（zero-base-tenant）是为了解决多租户环境下的数据隔离和资源共享问题而设计的。这些模块共同构成了我们系统的基础架构。

在业务模块方面，我们首先构建了系统业务模块（zero-system），它负责处理系统的核心业务逻辑。除了已经实现的模块外，我们的架构设计还预留了空间，以便未来可以灵活地添加更多业务模块，例如内容管理模块、财务管理模块等。这些模块将根据实际业务需求进行开发和集成，以支持系统的扩展性和可维护性。

4.5.2 系统模型设计

本研究采纳了分层结构设计，基于角色的访问控制（RBAC）权限模型构建。系统通过引入基础实体类 BaseEntity 和租户实体类 TenantEntity，实现了全面的审计功能和多租户支持。BaseEntity 类包含了创建者、创建时间、更新者、更新时间等与审计相关的字段，而 TenantEntity 类继承自 BaseEntity 并额外加入了租户标识字段，从而实现了多租户数据的隔离。

核心业务实体包括用户（User）、角色（Role）、部门（Dept）、岗位（SysPost）和菜单（Menu）。用户实体负责管理操作者信息，包括账号、密码、个人信息等；角色实体定义了权限组，通过角色字符标识不同的权限级别；部门实体采用了树形结构设计，支持多级组织架构管理；岗位实体用于细化人员职责的划分；菜单实体则负责系统功能和权限的具体配置。为处理实体间的多对多关联关系，系统设计了一系列中间表：用户-角色关联表（UserRole）实现用户的多角色配置，用户-岗位关联表（UserPost）支持用户多岗位设置，角色-菜单关联表（RoleMenu）负责实现细粒度的功能权限控制，角色-部门关联表（RoleDept）用于管理数据权限范围。这种关联设计既确保了数据关系的完整性，又提供了灵活的权限配置能力，具体关系可参考 ER 图（图 4-6-2）。

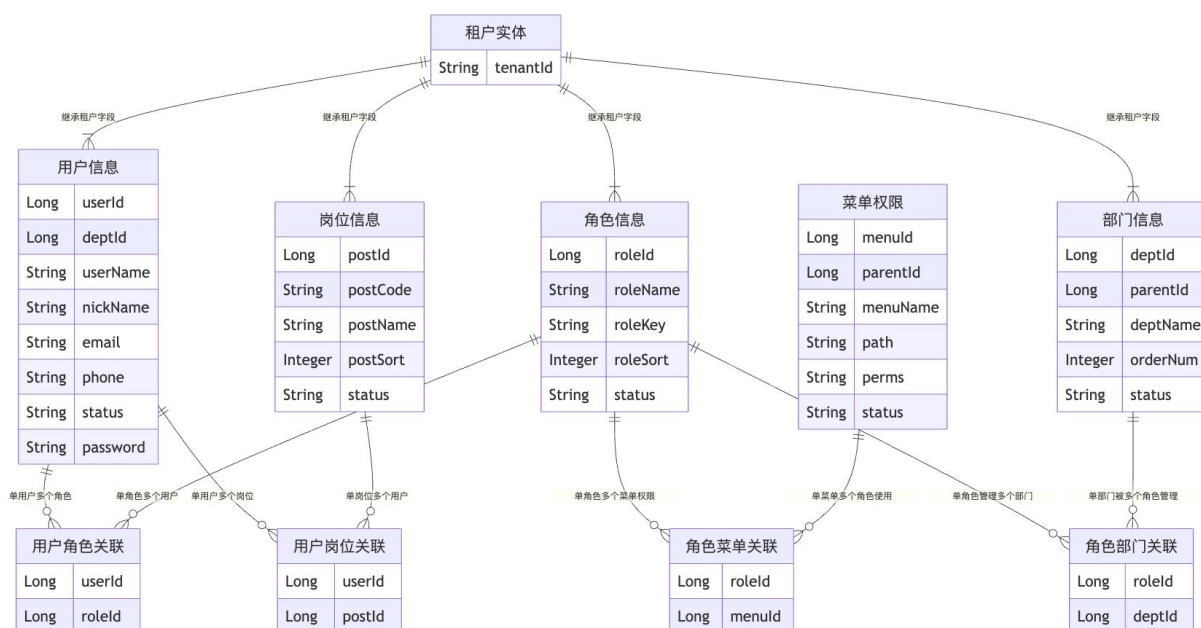


图 4-6-2 数据库实体关系 ER 图

该模型设计的优势体现在严格的权限控制、数据隔离、灵活的组织架构管理、完整的审计追踪能力及良好的可扩展性。通过统一的基础实体类和标准化命名，提升了代码的可维护性。此设计不仅满足企业级应用的安全性、扩展性和维护性需求，还为未来功能扩展和系统升级提供了坚实基础。

4.5.3 权限认证过程

首先，在认证层面，系统支持多种登录方式，包括账号密码登录和社交平台登录。用户登录时，系统会进行多重校验，包括客户端验证、租户验证和凭证校验。认证通过后，Sa-Token 会创建用户会话，生成 JWT 令牌^[8]，并将会话信息存储到多级缓存中（Caffeine + Redis），实现分布式会话管理。

在权限控制方面，系统基于 RBAC 模型实现了细粒度的权限管理。用户与角色之间、角色与权限之间均采用多对多关系，通过中间表实现灵活的权限分配。权限控制分为两个维度：角色权限 (Role) 和功能权限 (Permission)。使用 Sa-Token 的注解 (@SaCheckRole、@SaCheckPermission) 实现声明式权限控制，同时支持代码级的权限校验。

为了提升系统性能，认证信息的存储采用了多级缓存策略。本地缓存 (Caffeine) 用于提升高频访问的性能，Redis 缓存则确保分布式环境下的会话一致性。系统还实现了

完整的异常处理机制，针对未登录、无权限、角色失效等异常情况提供统一的处理方案。

在多租户场景下，系统通过租户上下文实现数据隔离，确保不同租户间的数据安全。同时，权限模型支持灵活的数据权限控制，可以基于部门、角色等维度进行数据访问控制。整个认证授权方案既保证了系统安全性，又兼顾了性能和扩展性，适合企业级应用的各种认证授权需求。具体认证过程参见时序图（图 4-6-3）。

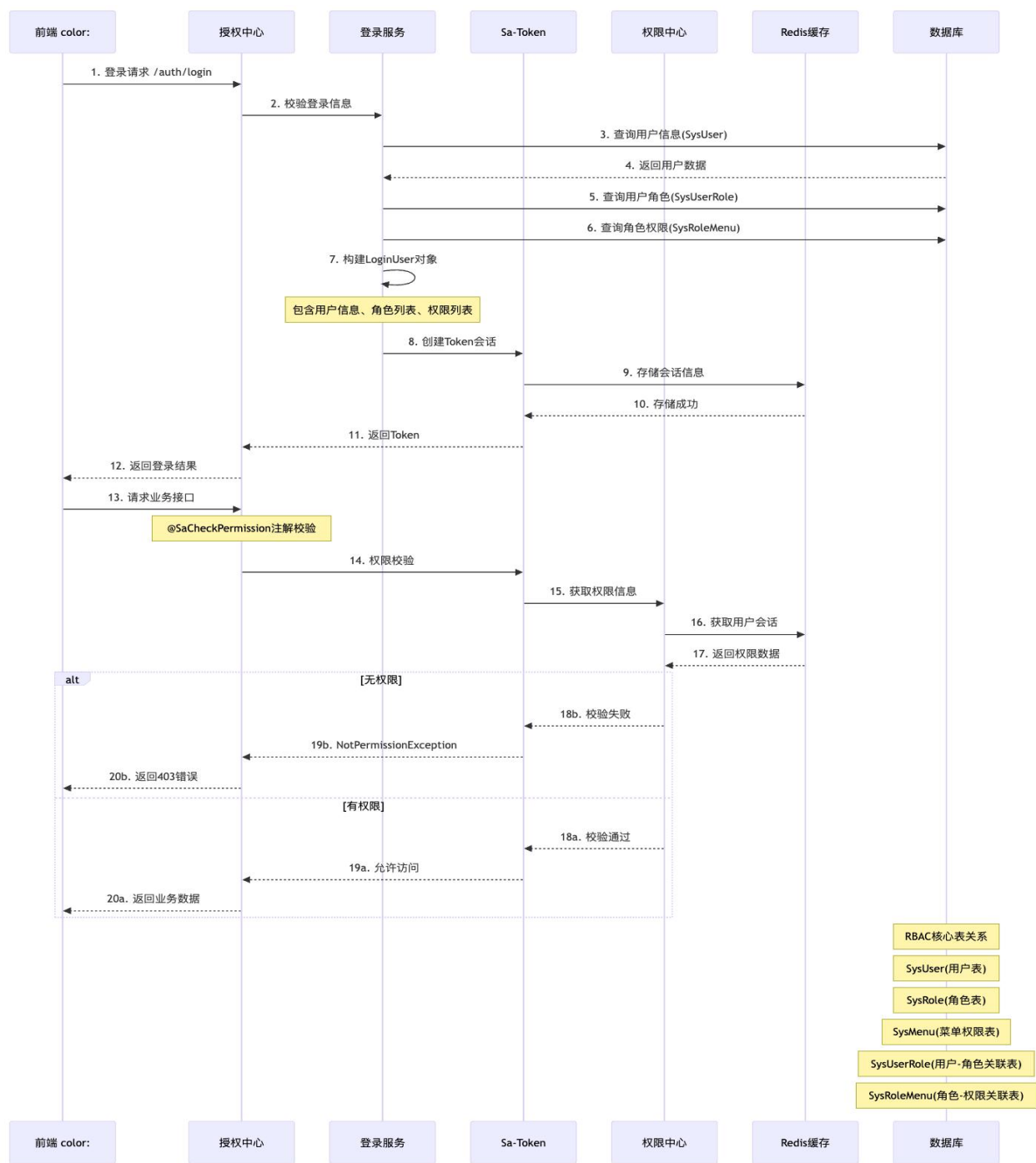


图 4-6-2 认证过程时序图

第五章 应用与实践

5.1 客户端实现

5.1.1 自动生成路由和布局

首先我们实现一个页面首先要在 `vue-router` 的路由配置中添加一个页面配置以及对页面路径的组件，这样的方式会添加冗余的信息，因为绝大多数场景都是和页面路径一一对应的。前面提到过软件开发中另一个原则：“约定大于配置”。我们可以通过目录自动生成路由的方式，这样就能像访问文件夹的方式访问对应页面，既能满足路由访问页面的需求，又能降低冗余配置造成的开发维护成本。

具体实现我们可以通过 `vite-plugin-pages` 插件来支持，具体配置如下：

```
Pages({
  dirs: [{ dir: 'src/pages', baseRoute: '/' }],
  exclude: ['**/_views/**', '**/components/**'],
  importMode: 'async',
  extendRoute(route) { return toMerged({ meta: { auth: true, layout: 'base' } }, route) },
})
```

`dirs[n].dir` 设置组件页面的根路径，`baseRoute` 定义基础路由。`exclude` 用于排除不需要路由的目录，如布局和公共组件目录。`importMode` 采用异步引入组件，提高性能。`extendRoute` 方法扩展路由，添加 `meta` 字段用于路由守卫和布局渲染。

使用 `vite-plugin-pages` 插件，按约定在 `src/pages` 目录组织组件文件，插件自动生成路由配置，简化路由管理，降低项目维护成本。

在路由守卫中，根据路由 `meta` 字段判断布局，如 `meta` 包含 `'base'` 则使用管理后台布局。动态渲染布局组件，并将页面组件作为子组件渲染，提升开发效率和维护便利性。

在必要时我们还可以在页面中重写，在上面代码中 `extendRoute` 我们使用了 `es-toolkit.toMerged` 函数优先读取在页面组件中自定义的配置，以登录页 `pages/login/index.vue` 为例我们在组件中加入配置：`<route lang="json5">{ meta: { layout:`

'blank', auth: false } }</route>这样即便在全局路由守卫中配置了默认的管理后台布局，登录页也会使用 blank 布局，无需跳转即可展示登录表单，提升用户体验。此外，通过自动生成路由和布局，我们能够快速响应需求变更，只需添加或修改页面组件，路由和布局将自动更新，无需手动调整路由配置，显著提高了开发效率和系统的可维护性。

最后布局的切换会根据路由配置自动切换，具体代码片段如下：

```
<template> <component :is="currentLayout" /></template>

<script setup lang="ts">

import BaseLayout from './layouts/BaseLayout.vue'
import BlankLayout from './layouts/BlankLayout.vue'

// # switch layout

const currentLayout = shallowRef(BaseLayout)

const layouts = { base: BaseLayout, blank: BlankLayout }

watchEffect(() => {

  const layoutName: string = route.meta?.layout as string

  currentLayout.value = layouts[layoutName || 'blank']

})

</script>
```

通过监听 route.meta.layout 的改变来切换在 layouts 目录中定义好的布局，这种布局切换机制确保了在不同页面间切换时，能够迅速应用相应的布局样式，进一步提升了用户界面的灵活性和用户体验。例如，在管理系统中，大多数页面可能采用统一的管理后台布局，包含侧边栏、导航栏和主内容区域，便于用户进行各项操作。然而，在登录页或 404 错误页等特定场景下，可能需要采用更为简洁的布局样式，以避免不必要的干扰，突出关键信息。此时，通过路由配置中的 meta 字段指定 layout 为'blank'，即可自动应用空白布局，无需额外的布局配置代码。

5.1.2 界面自适应实现

下面将以登录界面为案例，展示如何构建一个自适应的界面^[9]。本系统的前端架构融合了 Sass 预处理器与 UnoCSS 原子化 CSS 框架。借助这两者的协同作用，我们能够

以高效和便捷的方式，在开发流程中依据设计需求，实现基于媒体查询的响应式设计。

首先，我们需要在项目中引入 unocss/vite，并对 theme.breakpoints 进行设置，以定义响应式拦截断点。具体操作请参考以下代码：

```
UnoCSS({
  theme: { breakpoints: { sm: '640px', md: '768px', lg: '1024px', }, },
  transformers: [ transformerDirectives(), /* support @apply、@screen and theme()
*/ ],
})
```

可以看到上面配置代码中，设置了 sm、md 和 lg 三个断点，后面的 transformer 中提供了方便编码的 @screen 高级函数。比如我们的登录页右侧表单需要在非电脑端或者将电脑屏幕缩小到 lg:1024px 像素宽度时登录表单全屏展示时我们可以添加代码：

```
@screen lt-lg { @apply: min-w-full; }
```

5.1.3 深色浅色主题切换

主题切换的核心是借助 CSS Variables 即 var() 函数来实现，利用 Sass 预处理器、HTML 属性配合选择器来实现切换。在本项目中，主题切换的实现采用了 Vue 3 的组合式 API 和状态管理工具 Pinia，结合 @vueuse/core 提供的工具函数，实现了一个灵活且可维护的主题切换系统，具体效果如（图 5-1-2）所示。

具体实现方案如下：

第一步，主题状态管理：在 base.module.ts 中，使用 Pinia 存储主题相关的状态：

```
export const useBaseStore = defineStore('base', () => {
  const setting = reactive({
    theme: useLocalStorage<BaseTheme>('setting.theme', 'light'),
    setTheme(theme: BaseTheme) { setting.theme = theme },
  })
  return { setting }
})
```

第二步，主题切换逻辑：在 App.vue 中，通过监听主题状态的变化，动态设置 HTML

根元素的 data-theme 属性:

```
const theme = computed(() => setting.theme)

const systemTheme = computed(() => {
  return useMediaQuery('(prefers-color-scheme: dark)').value ? 'dark' : 'light'
})

watchImmediate( // 监听用户主题设置
  () => theme.value,
  () => {
    if (theme.value === 'auto')
      document.documentElement.setAttribute('data-theme', systemTheme.value)
    else document.documentElement.setAttribute('data-theme', theme.value)
  } )

watchImmediate(systemTheme, () => { // 监听系统主题变化
  if (theme.value !== 'auto') return
  document.documentElement.setAttribute('data-theme', systemTheme.value)
})
```

主题切换组件: 在 LayoutActions.vue 中, 实现了主题切换的用户界面:

```
const ThemeCheckTag = ({ text, value, icon }) => (
  <el-check-tag class='check-item' checked={setting.theme === value}
    onChange={() => setting.setTheme(value)}>
    <svg-icon v-show={icon} class='mr-12' name={icon} /> {t(text)}
  </el-check-tag>)
)
```

这种实现方案具有多个显著优点: 通过 Vue 3 的响应式系统和 Pinia 状态管理实现了可靠的响应式设计, 支持跟随系统主题自动切换以提供良好的用户体验; 同时利用 useLocalStorage 实现了主题设置的持久化存储, 确保页面刷新后主题设置不会丢失; 在技术实现上, 通过 CSS Variables 和 Sass 预处理器提供了良好的可扩展性, 使得添加新的主题变量和样式变得简单; 此外, 使用 CSS Variables 进行主题切换的方式避免了重复加载样式文件, 在保证功能实现的同时也优化了系统性能。

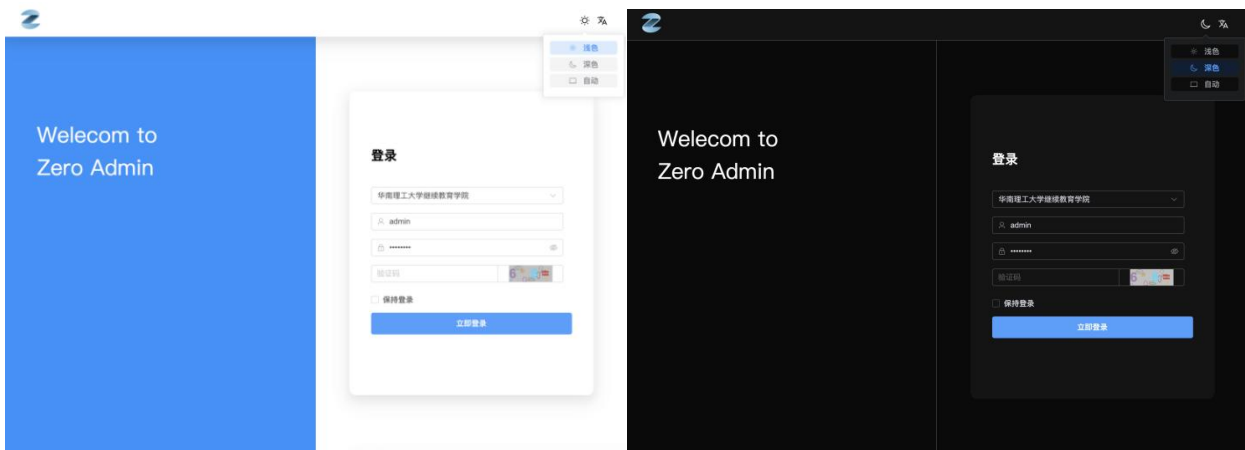


图 5-1-3 亮色主题和深色主题

5.1.4Element Plus 组件二次封装

前面已经介绍过对 Element Plus 组件二次封装的必要性,下面通过对 el-input 二次封装成 ze-input 组件的源码直观地介绍一下本系统二次封装的原则和方法 (图 5-1-4)。

```
src > components > ZeInput.vue > ...
1  <template>
2    <el-input v-bind="omit(mergeProps($attrs, props), ['prefixIcon', 'suffixIcon'])" class="ze-input" ref="rawRef">
3      <template #prefix v-if="props.prefixIcon"><svg-icon :name="props.prefixIcon" /></template>
4      <template #suffix v-if="props.suffixIcon"><svg-icon :name="props.suffixIcon" /></template>
5      <template v-for="(, name) in $slots" #[name]="scope">
6        <slot :name="name" v-bind="scope" />
7      </template>
8    </el-input>
9  </template>
10 <script setup lang="ts">
11 > import type { InputProps } from 'element-plus'
12 type ElInputType = InputInstance
13 type ZeInputProps = Partial<InputProps> & { prefixIcon?: string; suffixIcon?: string }
14 const props = withDefaults(defineProps<ZeInputProps>(), {
15   clearable: true,
16 })
17 const rawRef = ref<ElInputType>()
18 defineExpose<ElInputType>(new Proxy( {},
19   {
20     get: (_target, prop) => rawRef.value?.[prop],
21     has: (_target, prop) => prop in (rawRef.value || {}),
22   },) as ElInputType, )
23 </script>
24 <style lang="scss" scoped>
25 .ze-input {
26   min-width: 160px;
27   :deep(.el-input-group__prepend) { background-color: unset; }
28   :deep(.el-input-group__append) { background-color: unset; }
29 }
30 </style>
```

图 5-1-4 ZeInput.vue 组件源码

通过 defineProps 定义组件属性,并用 withDefaults 为属性设定默认值,确保组件在未指定属性时有合理行为,同时允许用户覆盖默认值。

属性整合与筛选：使用 `mergeProps` 和 `omit` 函数，可有效整合并筛选组件属性，确保接收属性既全面又精确，避免多余传递。

插槽的高效运用：通过 `v-for` 循环遍历 `$slots`，动态生成插槽内容，赋予组件灵活处理插槽的能力，从而提升其适应性和可扩展性。

内部实例的开放：通过 `defineExpose` 函数，使组件内部的方法和属性得以对外公开，便于外部组件进行访问和操作，这对于需要与外部组件交互的封装组件尤为关键。

样式的个性化定制：通过使用 `:deep()` 选择器函数，实现对 Element Plus 组件内部样式的个性化定制，确保样式的有效覆盖和定制化，以符合项目设计规范的要求。

5.1.5 封装 Hooks: useApi

在现代前端开发中特别是 Vue3 和 React 中，Hooks 是一种用于在函数组件中重用状态逻辑的手段。通过构建 Hooks，可以将繁杂的逻辑简化为简单的函数，增强代码的可读性和重用性。`useApi` 是一个定制的 Hook，用于封装 API 调用逻辑。它通过接受 API 函数、调用参数和配置项，提供了统一的接口来处理 API 调用、操控调用状态（如加载状态）、处理成功和错误回调等。`useApi` 的封装不仅简化了 API 调用的运用，还提升了代码的可维护性和统一性，使得在不同组件中重用 API 调用逻辑变得更加简单。

```
type UseApiOnSuccessFn<T> = (res?: _AxiosResponse<ApiResponse<T>>) => void
type UseApiOnSubmitFn<T, X = T & { [prop: string]: any }> = (data: X)
=>Promise<boolean | X>

export type UseApiOnErrorFn = (err: ApiError) => void

type ReturnFields<D> = [
  {data: Ref<D | undefined>}, { request: Function }, { loading: Ref<boolean> },
]

export function useApi<P, D>(  
  api: (params: P) => ApiPromise<D>,  
  params?: (Partial<P> | any) | (() => Partial<P> | any),  
  options?: {  
    immediate?: boolean
```

```

tipError?: boolean | string | Ref
tipSuccess?: boolean | string | Ref
onSuccess?: UseApiOnSuccessFn<D>
onError?: UseApiOnErrorFn
onFinally?: () => void
onSubmit?: UseApiOnSubmitFn<P>
},
): UseApiReturn<D>

```

从上面代码的方法类型定义就不难看出 hooks 的封装难度，但是一旦我们回到使用者的角度来看，就会发现 hooks 的强大威力！以登录接口调用为例，我们只需要很简单的代码就能完成接口的调用，具体代码片段如下：

```

const loginForm = reactive<LoginForm>({ username: "", password: "" })
const [userInfo, fetchLogin] = useApi(login, loginForm)
<ze-button @click="fetchLogin">

```

通过上述代码，我们创建了一个登录表单，并将其绑定到 fetchLogin 函数。当用户点击登录按钮时，fetchLogin 函数将被触发，调用登录接口，并根据返回结果执行相应的操作。useApi Hook 封装了 API 调用的细节，使得调用过程简洁明了，降低了出错的可能性，提高了代码的可读性和可维护性。

在 useApi Hook 中，immediate 选项允许我们指定是否在组件挂载时立即执行 API 调用。当 immediate 设置为 true 时，组件挂载后将立即触发 API 调用，无需手动触发。这对于需要在组件加载时立即获取数据的场景非常有用，如获取用户信息、初始化数据列表等。通过合理配置 immediate 选项，我们可以更灵活地控制 API 调用的时机，满足不同的业务需求。

5.1.6 定义接口层 fetch 接口访问

在企业级管理系统中，接口请求的规范化和统一化管理是保证系统稳定性和可维护性的关键因素。本系统基于 Axios 封装了一套完整的接口请求解决方案，并以用户管理模块为例，展示了接口请求的实践应用。

在用户管理模块（user.api.ts）中，系统采用了模块化的方式组织接口，统一管理用户相关的接口请求。每个接口函数都经过类型定义，确保了类型安全和代码提示。例如，用户列表查询接口：

```
export function listUser(query: UserQuery): ApiPromisePage<UserVO> {  
    return fetch({ url: '/system/user/list', method: 'get', params: query })  
}
```

系统通过 TypeScript 的类型系统，定义了完整的接口请求和响应类型。如 UserQuery 定义了查询参数的结构，UserVO 定义了返回数据的格式，这种强类型的设计大大提高了代码的可维护性和开发效率。同时，系统还定义了统一的响应格式 ApiResponse<T> 和分页数据结构 ApiPage<T>，实现了对请求响应的规范化处理。

```
export interface ApiPageForm { /** 分页查询表单 */  
    pageNo?: number  
    pageSize?: number }  
  
export class ApiPagination implements ApiPageForm { /** 分页数据结构 */  
    pageNo!: 1  
    pageSize!: 10  
    total!: 0 | number  
    totalPage?: 0 | number }  
  
export interface ApiResponse<T> { /** 后端接口返回数据结构 */  
    code: number | string  
    msg: string  
    data: T }  
  
export class ApiPage<T> extends ApiPagination { 分页数据体 */  
    rows!: Array<T> }  
  
export type ApiPromise<T> = AxiosPromiseE<ApiResponse<T>, ApiError>  
export type ApiPromisePage<T> = AxiosPromiseE<ApiResponse<ApiPage<T>>, ApiError>
```

在请求拦截器中，系统统一处理了认证信息、语言设置等通用请求头，确保了接口调用的安全性和国际化支持。响应拦截器则统一处理了响应状态码、错误信息等，实现

了统一的错误处理机制。特别是对于 401 未授权等特殊状态码，系统会自动处理登录失效的情况，提升了用户体验。

该封装方案不仅提供了类型安全和代码提示，还实现了统一的错误处理和状态管理，大大提高了开发效率和代码质量。通过这种方式，开发人员可以专注于业务逻辑的实现，而不需要过多关注接口调用的细节处理。

5.1.7 一百行实现增删查改

在现代企业级管理系统开发中，组件的封装和复用是提高开发效率的关键。本项目基于 Vue 3 和 Element Plus，通过一系列的组件封装和自定义 Hooks，实现了高效的企业级管理系统开发方案。

从组件封装的角度来看，项目实现了以 ze- 为前缀的一系列业务组件。其中，ze-form 组件通过配置化的方式生成表单，支持多种表单项类型和验证规则；ze-table 组件在 Element Plus 的表格组件基础上扩展了列过滤、固定列等功能；ze-actions 组件则统一了操作按钮的展示和处理逻辑。这些组件的封装大大减少了重复代码，提高了开发效率。

在 Hooks 封装方面，项目提供了 useTable、useFormuseApi 和 useModal 等核心 Hooks。useTable 封装了表格数据的加载和分页逻辑；useForm 提供了表单的生成和验证功能；useApi 统一处理了接口请求和响应；useModal 则封装了弹窗的显示和数据处理逻辑。这些 Hooks 的组合使用，详见（图 5-1-7），使得开发人员可以快速实现 CRUD 等常见业务场景。

以用户管理页面为例，其构建展示了组件与 Hooks 的协作：首先，利用 useForm 创建搜索和编辑表单，设定表单项和验证规则；接着，通过 useTable 加载表格数据并实现分页；然后，使用 useApi 处理增删改查等请求；最后，通过 useModal 展示编辑弹窗并提交数据。整个流程中，组件和 Hooks 紧密配合，构成了一个高效的数据管理循环。

组件化和 Hooks 化开发模式提高了代码复用性和可维护性，为开发人员设定了标准化范式。TypeScript 的类型检查和代码提示功能增强了代码质量和开发效率。这些策略特别适合企业级管理系统的快速开发和迭代。

综上所述，通过对接口层的封装，我们能够迅速地对服务器接口进行请求，在前后端分离的开发过程中将游刃有余。通过封装 Element Plus 和 hooks，以及运用 el-toolkit

和 vueuse 的工具库，我们可以在大约一百行代码内实现一个复杂的增删查该页面。这在企业开发过程中，能够显著提升代码质量并降低维护成本。还能更方便地借助 AI 人工智能提示代码或者进一步封装实现一个低代码的拖拉拽的开发平台。具体实现可参考项目 Gitee 或 Github 开源项目 vue-zero-admin。

```
1 <template>
2 <VPage>
3
4 <template #header>
5 <ze-form v-model="searchForm" :items="searchFormItems" ref="searchFormRef" inline>
6 <ze-form-item>
7 <ze-actions>
8 />
9 </ze-form-item>
10 <ze-form-item class="ml-a">
11 <ze-actions>
12 :actions="[ { icon: 'el-plus', content: '新增', type: 'primary', onClick: () => editRef.open() } ]"
13 />
14 </ze-form-item>
15 </ze-form>
16 </template>
17 <!-- 分页表格 -->
18 <ze-table>
19 :data="tableData"
20 :loading="loading"
21 :columns="[
22 { prop: 'id', hidden: true },
23 { type: 'index', label: 'base.index', align: 'center', width: '60px', fixed: 'left' },
24 { prop: 'name', label: 'doc.col.name', minWidth: 150, fixed: true },
25 ]"
26 :filterColVR="filterColRef"
27 >
28 <template #col-hasPassion="scope">
29 <el-switch :modelValue="scope.row.hasPassion" @update:modelValue="scope.row.hasPassion = $event" />
30 </template>
31 <ze-table-column min-width="180" fixed="right">
32 <template #default="scope">
33 <ze-actions>
34 :actions="[
35 { content: 'base.edit', text: true, onClick: () => editDlgRef.open(scope.row) },
36 { content: '配置', text: true, onClick: () => {} },
37 { content: 'base.delete', text: true, confirm: true, onClick: () => handleDel(scope.row) },
38 ]"
39 ellipsis
40 />
41 </template>
42 </ze-table-column>
43 </ze-table>
44 <!-- 分页 -->
45 <template #content-footer><ze-pagination class="ml-a" v-model="pagination" /></template>
46 </VPage>
47 <!-- 编辑弹窗 -->
48 <editDialog> <ze-form v-model="userForm" :items="userFormItems" :rules="userFormRules"></ze-form></editDialog>
49 </template>
50
51
52
53
54
55
56
57
58
59 <script setup lang="ts">
60 const searchFormRef = ref<ZeFormInstance>()
61 // 搜索表单
62 const searchForm = reactive({ name: '', pageNo: 1, pageSize: 10 })
63 // 表格分页查询
64 const [tableData, refresh, pagination, loading] = useTable(demoApi.list, searchForm, { immediate: true })
65
66 const filterColRef = ref()
67 const isEdit = computed(() => !isEmpty(userForm?.value?.id))
68 const formTitle = computed(() => (isEdit?.value ? '编辑' : '新增'))
69 // 生成表单
70 const [userForm, userFormItems, userFormRules] = useForm({
71 id: { value: '', item: { type: 'hidden' } },
72 name: {
73 value: '',
74 item: { type: 'text', label: '名字' },
75 rule: [{ required: true, message: '必填' } ] },
76 })
77 // 新增修改接口提交
78 const { request: fetchEdit, loading: submitting } = useApi({
79 (data: UserForm) => (isEdit.value ? demoApi.update(data) : demoApi.create(data)),
80 toReactive(userForm),
81 {
82 Codeium: Refactor | Explain | Generate Function Comment | X
83 onSuccess: () => {
84 editDlgRef.value.close()
85 refresh()
86 },
87 tipSuccess: computed(() => (isEdit.value ? '保存成功' : '新增成功')),
88 tipError: computed(() => (isEdit.value ? '保存失败' : '新增失败')),
89 },
90 })
91 // 新增修改弹窗
92 const [editDlgRef, EditDialog] = useModal({
93 title: formTitle,
94 onConfirm: () => fetchEdit(),
95 submitting,
96 })
97 // 删除
98 Codeium: Refactor | Explain | X
99 const handleDel = (row) => baseApi.delType(row).then(() => refresh() && ElMessage.success('删除成功'))
100 </script>
```

a) Template 部分

b)Script 部分

图 5-1-7 100 行实现 CRUD

5.2 服务端实现

5.2.1 模块拆分实现

系统采用分层架构设计，基于 Spring Boot 3.4.2 框架开发。整体分为基础架构层（zero-base）、业务模块层（zero-modules）和 Web 应用层（zero-admin-web）三个主要部分。基础架构层包含了多个独立的功能模块，如核心工具类（zero-base-core）、Web

服务基础功能（zero-base-web）、缓存服务（zero-base-redis）、数据访问层（zero-base-mybatis）、认证授权（zero-base-satoken）等。业务模块层主要包含系统管理模块（zero-system），负责处理用户、角色、权限等核心业务功能。Web 应用层作为系统入口，集成了 Spring Boot Admin Client 进行系统监控。系统使用 PostgreSQL 作为数据库，集成了多项先进技术，如 Sa-Token 和 JWT 用于安全认证，Redisson 和 Caffeine 用于缓存管理，MyBatis-Plus 用于数据访问等。整个项目采用组件化设计思想，各模块职责明确，依赖关系清晰，支持多租户、统一的数据处理（加密、脱敏、翻译）和分布式特性，同时集成了多种开发效率工具，便于扩展和维护，

5.2.2 数据库实现

下面列举出本系统多个核心表的详细设计，总体设计可以参考 UML 图（图 5-2-2）：

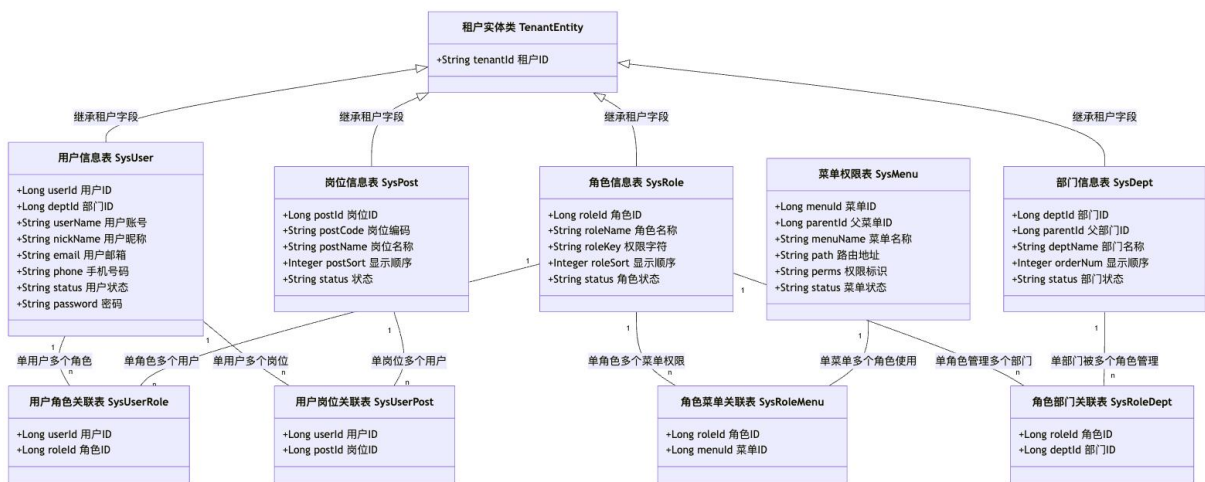


图 5-2-2 核心 UML 图

表 5-2-1 sys_tenant 租户表

字段名	字段类型	字段描述
id	bigint(20)	id
tenant_id	varchar(20)	租户编号
contact_user_name	varchar(20)	联系人

contact_phone	varchar(20)	联系电话
company_name	varchar(50)	企业名称
license_number	varchar(30)	统一社会信用代码
address	varchar(200)	地址
intro	varchar(200)	企业简介
domain	varchar(200)	域名
remark	varchar(200)	备注
package_id	bigint(20)	租户套餐编号
expire_time	datetime	过期时间
account_count	int	用户数量 (-1 不限制)
status	char(1)	租户状态 (0 正常 1 停用)

表 5-2-2 sys_user 用户表

字段名	字段类型	字段描述
user_id	bigint(20)	用户 ID
tenant_id	varchar(20)	租户编号
dept_id	bigint(20)	部门 ID
user_name	varchar(30)	用户账号
nick_name	varchar(30)	用户昵称
user_type	varchar(10)	用户类型 (sys_user 系统用户)
email	varchar(50)	用户邮箱
phonenumber	varchar(11)	手机号码
sex	char(1)	用户性别 (0 男 1 女 2 未知)
avatar	bigint(20)	头像地址
password	varchar(100)	密码
status	char(1)	账号状态 (0 正常 1 停用)
login_ip	varchar(128)	最后登录 IP

login_date	datetime	最后登录时间
remark	varchar(500)	备注

表 5-2-3 sys_dept 部门表

字段名	字段类型	字段描述
dept_id	bigint(20)	部门 id
tenant_id	varchar(20)	租户编号
parent_id	bigint(20)	父部门 id
ancestors	varchar(500)	祖级列表
dept_name	varchar(30)	部门名称
dept_category	varchar(100)	部门类别编码
order_num	int(4)	显示顺序
leader	bigint(20)	负责人
phone	varchar(11)	联系电话
email	varchar(50)	邮箱
status	char(1)	部门状态 (0 正常 1 停用)

表 5-2-4 sys_role 角色表

字段名	字段类型	字段描述
role_id	bigint(20)	角色 ID
tenant_id	varchar(20)	租户编号
role_name	varchar(30)	角色名称
role_key	varchar(100)	角色权限字符串
role_sort	int(4)	显示顺序
data_scope	char(1)	数据范围: 1. 全部数据权限 2. 自定义数据权限 3. 本部门的数据权限 4. 本部门及以下数据权限

menu_check_strictly	tinyint(1)	菜单树选择项是否关联显示
dept_check_strictly	tinyint(1)	部门树选择项是否关联显示
status	char(1)	角色状态 (0 正常 1 停用)
remark	varchar(500)	备注

表 5-2-5 sys_meny 菜单表

字段名	字段类型	字段描述
menu_id	bigint(20)	菜单 ID
menu_name	varchar(50)	菜单名称
parent_id	bigint(20)	父菜单 ID
order_num	int(4)	显示顺序
path	varchar(200)	路由地址
component	varchar(255)	组件路径
query_param	varchar(255)	路由参数
is_frame	int(1)	是否为外链 (0 是 1 否)
is_cache	int(1)	是否缓存 (0 缓存 1 不缓存)
menu_type	char(1)	菜单类型 (M 目录 C 菜单 F 按钮)
visible	char(1)	显示状态 (0 显示 1 隐藏)
status	char(1)	菜单状态 (0 正常 1 停用)
perms	varchar(100)	权限标识
icon	varchar(100)	菜单图标

表 5-2-6 sys_use_role 关系表

字段名	字段类型	字段描述
user_id	bigint(20)	用户 ID
role_id	bigint(20)	角色 ID

表 5-2-7 sys_role_menu 关系表

字段名	字段类型	字段描述
role_id	bigint(20)	角色 ID
menu_id	bigint(20)	菜单 ID

表 5-2-8 sys_role_dept 关系表

字段名	字段类型	字段描述
role_id	bigint(20)	角色 ID
dept_id	bigint(20)	部门 ID

表 5-2-8 sys_client 客户端表

字段名	字段类型	字段描述
client_id	varchar(64)	客户端 id
client_key	varchar(32)	客户端 key
client_secret	varchar(255)	客户端密钥
grant_type	varchar(255)	授权类型
device_type	varchar(32)	设备类型
active_timeout	int(11)	token 活跃超时时间
timeout	int(11)	token 固定超时
status	char(1)	状态 (0 正常 1 停用)

5.2.3 认证流程实现

本文将对项目的核心流程进行深入探讨，重点聚焦于登录认证、访问认证、多级缓存策略、权限验证机制以及异常处理机制。在登录认证流程中，系统通过验证用户凭证生成 JSON Web Token (JWT)，并将用户信息存储于多级缓存系统中，随后向客户端返回该 Token 以完成认证过程。访问认证流程则利用 Sa-Token 框架对 Token 的有效性进行校验，并结合角色权限检查 (@SaCheckRole) 与操作权限检查 (@SaCheckPermission)，实现精细化的访问控制。为提高认证信息检索的效率，本项目采用 Caffeine 作为一级本地缓存，Redis 作为二级分布式缓存，构建了高效的多级缓存架构。权限验证机制不仅支持角色级别和功能级别的权限控制，还实现了多租户权限隔离以及灵活的权限组合验证策略，以适应复杂业务场景的需求。此外，本研究设计了一套统一的异常处理机制，包括标准化的错误响应、详尽的日志记录以及全流程的异常管理，确保从用户登录、权限校验到注销等环节的稳定性和可维护性。关于用户登录、权限校验到注销过程中的一些处理细节和具体实现：具体流程可参考流程图（图 5-2-3）

1. Sa-Token 的核心配置，通过 SaTokenConfig 实现基础配置：

@AutoConfiguration

```
public class SaTokenConfig {  
    @Bean public StpLogic getStpLogicJwt() {  
        return new StpLogicJwtForSimple();// JWT 简单模式, 实现 token 的生成和验证  
    }  
    @Bean public StpInterface stpInterface() {  
        return new SaPermissionImpl();// 注入权限验证接口, 实现 RBAC 权限控制  
    }  
    @Bean public SaTokenDao saTokenDao() {  
        return new PlusSaTokenDao();// 自定义多级缓存, 提升性能  
    }  
}
```

2. 权限验证的核心逻辑通过实现 StpInterface 来完成：

```
public class SaPermissionImpl implements StpInterface {  
    @Override  
    public List<String> getPermissionList(Object loginId, String loginType) {  
        // 获取用户的菜单权限列表  
        LoginUser loginUser = LoginHelper.getLoginUser();  
        if (userType == UserType.SYS_USER)  
            return new ArrayList<>(loginUser.getMenuPermission());  
        return new ArrayList<>();  
    }  
    @Override  
    public List<String> getRoleList(Object loginId, String loginType) {  
        // 获取用户的角色权限列表  
        LoginUser loginUser = LoginHelper.getLoginUser();  
        if (userType == UserType.SYS_USER)
```

```

        return new ArrayList<>(loginUser.getRolePermission());
    }
    return new ArrayList<>();
}
}

```

3. 系统采用多级缓存策略，通过实现 SaTokenDao 来完成：

```

public class PlusSaTokenDao implements SaTokenDao {
    // Caffeine 本地缓存配置

    private static final Cache<String, Object> CAFFEINE = Caffeine.newBuilder()
        .expireAfterWrite(5, TimeUnit.SECONDS)
        .initialCapacity(100).maximumSize(1000).build();

    @Override public String get(String key) {
        // 多级缓存查询：先查 Caffeine，未命中则查 Redis
        Object o = CAFFEINE.get(key, k -> RedisUtils.getCacheObject(key));
        return (String) o;
    }

    // 其他缓存操作方法.....
}

```

4. 在具体的业务接口中，通过注解实现权限控制

```

@RestController
@RequestMapping("/system/menu") // 接口 path
public class SysMenuController extends BaseController {
    @SaCheckRole(TenantConstants.SUPER_ADMIN_ROLE_KEY)
    @SaCheckPermission("system:menu:edit") // 操作权限 key
    @PutMapping
    public R<Void> edit(@Validated @RequestBody SysMenuBo menu) {}
}

```

5. 异常处理机制：

```

@RestControllerAdvice

```

```
public class SaTokenExceptionHandler {  
    @ExceptionHandler(NotLoginException.class)  
    public R<Void> handleNotLoginException(NotLoginException e, HttpServletRequest  
    request) {  
        return R.fail(HttpStatus.HTTP_UNAUTHORIZED, "认证失败, 无法访问系统资源");  
    }  
}
```

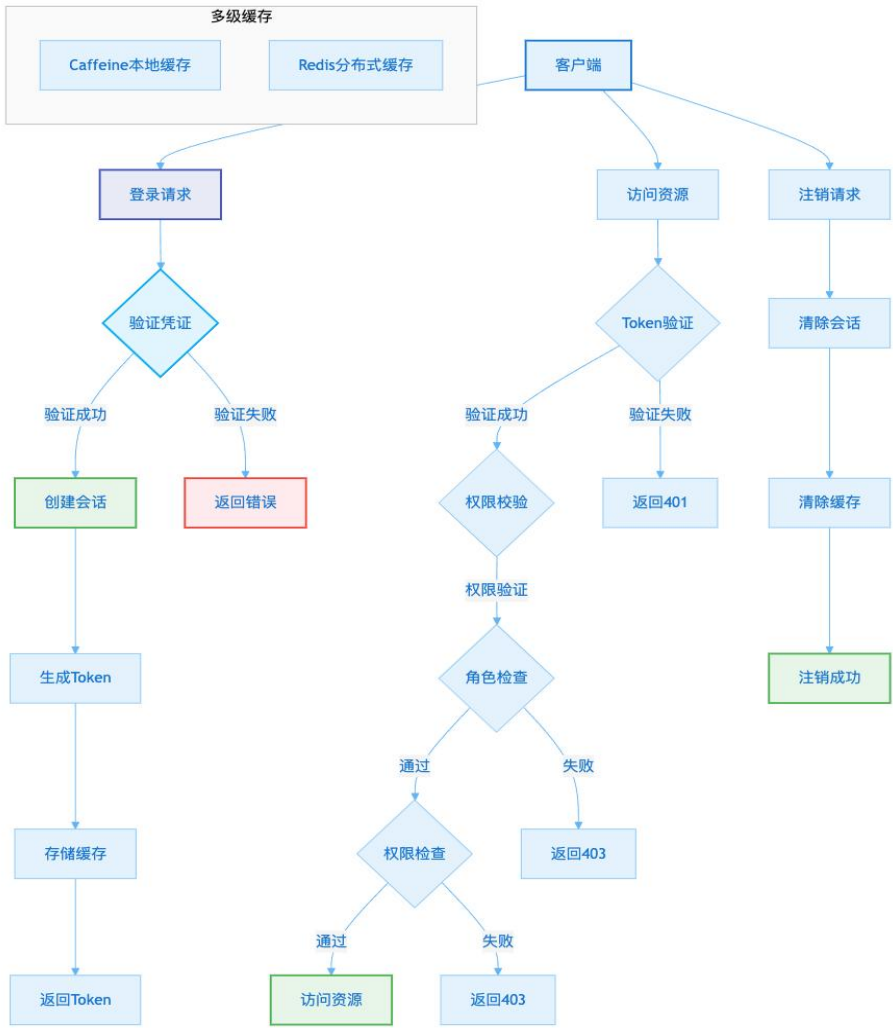


图 5-2-3 认证流程图

5.2.4 功能实现

为更直观地展示本企业信息管理系统的功能布局及操作流程，以下依次列出各核心模块的界面预览截图，包括租户管理、用户管理、岗位管理、部门管理、菜单配置、角色权限分配以及数据字典维护等关键功能界面。这些截图不仅有助于快速了解系统的整

体架构和交互设计，还能为后续的系统测试工作提供清晰的参考依据，确保测试用例的覆盖率和准确性。



图 5-2-1 租户管理



图 5-2-2 用户管理

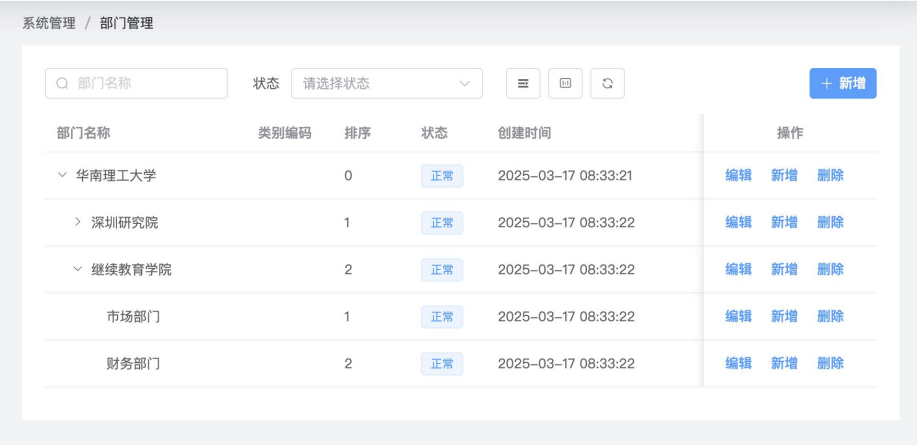


图 5-2-3 部门管理

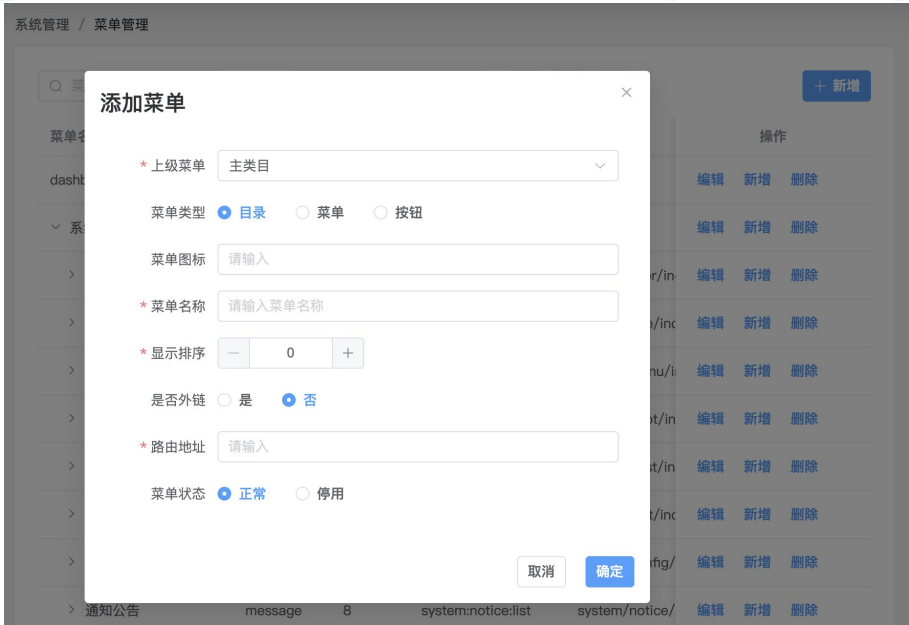
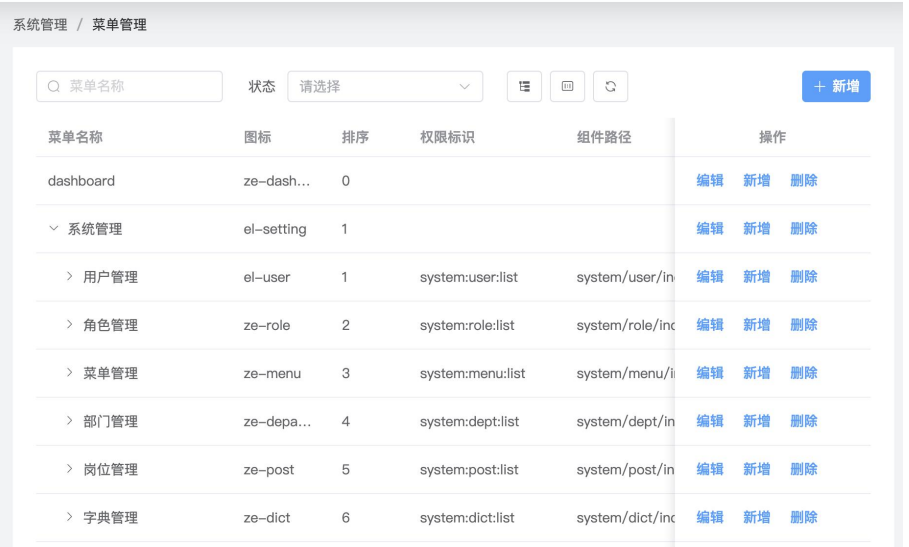


图 5-2-4 菜单管理



图 5-2-5 角色管理

第六章 测试与部署

6.1 软件测试

6.1.1 功能测试

功能测试是对系统各项功能进行全面验证的过程，目的是确保系统能够按照需求规格正常运作。测试工作主要包含以下几个关键环节：

- a) 测试准备：基于需求文档和技术方案，制定详细的测试计划和测试用例。
- b) 功能验证：按照测试用例逐一检查系统功能，确保各项功能符合预期。
- c) 全面检测：除核心功能外，还需测试用户界面、数据处理、兼容性和安全性等。
- d) 问题修复：通过测试发现潜在缺陷并及时修复，提升系统稳定性。
- e) 用户体验：从最终用户角度出发，验证系统的易用性和实用性。

在前期的模块设计中，我们已经明确了系统需要实现的具体功能。这些功能的实现情况和测试结果将直接反映系统是否真正满足用户需求，是项目验收的重要依据。

6.1.2 测试用例

为确保系统功能符合需求规格，我们针对各功能模块设计了详细的测试方案，具体测试用例如下：

表 6-1-1 登录界面测试

用例编号	Auth001	功能模块	认证模块
测试目的	验证正常登录流程		
前置条件	系统正常运行，网络连接正常，登录页面正常渲染		
步骤	操作描述	数据	期望结果
1	输入用户名	admin	输入框校验通过
2	输入密码	admin123	输入框校验通过
3	输入验证码	根据图片内容输入	验证码输入框显示输入内容
4	点击登录按钮	-	1. 显示加载动画 2. 发送登录请求 3. 获取 Token 4. 跳转到首页

表 6-1-2 登录表单测试

用例编号	Auth002	功能模块	认证模块
测试目的	客户端验证表单校验		
前置条件	系统正常运行，网络连接正常，登录页面正常渲染		
步骤	操作描述	数据	期望结果
1	在表单为空时点击登录	-	1.表单校验不为空提示正常显示 2.异常 toast 正常提示
2	输入用户名	admin	失焦后对应输入框校验提示消失
3	输入密码	admin123	失焦后对应输入框校验提示消失
4	输入验证码	1	失焦后对应输入框校验提示消失

表 6-1-3 登录失败测试

用例编号	Auth003	功能模块	认证模块
测试目的	服务端表单验证和登录失败验证		
前置条件	系统正常运行，网络连接正常，登录页面正常渲染		
步骤	操作描述	数据	期望结果
1	输入用户名	admin	输入框校验通过
2	输入密码	wrong_pass	输入框校验通过
3	输入验证码	根据图片内容输入	验证码输入框显示输入内容
4	点击登录按钮	-	1. 显示加载动画 2. 发送登录请求 3. 提示”密码输入错误{n}次” 4. 验证码图片重新获取

表 6-1-4 注销登录测试

用例编号	Auth004	功能模块	认证模块
测试目的	检测注销登录流程		
前置条件	正常进入系统，并已正常登录		
步骤	操作描述	数据	期望结果
1	点击右上角用户菜单退出	-	1. 请求注销接口 2. 清空登录用户数据 3. 返回登录页面

表 6-1-5 租户新增测试

用例编号	Tenant001	功能模块	租户模块
测试目的	测试租户管理是否正常		
前置条件	正常进入系统，并已正常登录		
步骤	操作描述	数据	期望结果
1	点击菜单租户管理	-	1. 成功进入租户管理 2. 显示出租户分页列表
2	点击新增	-	新增弹窗弹出并展示表单
3	依次填写表单内容	企业名称: 华南理工继续教育学院 用户名: hnlg 用户密码: hnlgl23 用户数量: -1	1. 成功调用新增租户接口 2. 新增成功后关闭弹窗 3. 新增成功后刷新列表

表 6-1-6 部门新增测试

用例编号	Dept001	功能模块	部门模块
测试目的	测试部门管理新增部门		
前置条件	正常进入系统，并已正常登录		
步骤	操作描述	数据	期望结果
1	点击菜单系统管理/部门管理	-	成功进入界面，并显示新增按钮
2	点击新增按钮	-	弹出新增部门表单弹窗
3	填写部门新信息	选择上级部门 部门名称: 华南理工大学	1. 上级部门列表正常显示 2. 表单校验不异常
4	点击确认	-	1. 保存成功 2. 显示成功提示 3. 列表更新

表 6-1-7 部门编辑功能测试

用例编号	Dept002	功能模块	部门模块
测试目的	验证部门信息编辑功能		
前置条件	部门列表中存在可编辑的部门数据		
步骤	操作描述	数据	期望结果
1	点击"编辑"按钮	-	弹出编辑表单，显示原有数据
2	修改部门信息	部门名称: 修改后名	表单显示修改后的内容

3	点击保存按钮	-	3. 保存成功 4. 显示成功提示 3. 列表更新
---	--------	---	---------------------------------

表 6-1-8 部门搜索功能测试

用例编号	Dept003	功能模块	部门模块
测试目的	验证部门搜索功能		
前置条件	部门列表中存在多条数据		
步骤	操作描述	数据	期望结果
1	输入部门名称	华南	搜索框显示输入内容
2	选择部门状态	正常	下拉框显示选中状态
3	等待自动触发搜索	-	列表显示符合条件的数据
4	点击重置按钮	-	1. 清空搜索条件 2. 恢复列表原始数据

表 6-1-9 部门删除测试

用例编号	Dept004	功能模块	部门模块
测试目的	验证部门删除功能		
前置条件	部门列表中存在多条数据		
步骤	操作描述	数据	期望结果
1	点击"删除"按钮	-	弹出二次确认框"确定要删除吗?"
2	点击"确定"按钮	-	1. 请求删除接口 2. 如果该列存在下级部门则提示 3. 如果不存在则正确删除 4. 刷新列表, 不再显示删除列

表 6-1-10 菜单树形展示测试

用例编号	Menu001	功能模块	菜单模块
测试目的	验证菜单树形结构的正确展示		
前置条件	用户已登录系统且具有菜单管理权限		
步骤	操作描述	数据	期望结果
1	访问菜单管理页面	-	显示树形菜单表格
2	验证表格列信息	-	显示所有必要列: 菜单名称、图标、排序、权限标识、组件路径、状态、创建时间

3	点击展开/折叠按钮	-	正确展开/折叠树形结构
4	点击列过滤按钮	-	显示列显示/隐藏配置面板

表 6-1-11 菜单新增功能测试

用例编号	Menu002	功能模块	菜单模块
测试目的	验证新增不同类型菜单的功能		
前置条件	已打开菜单管理页面		
步骤	操作描述	数据	期望结果
1	点击新增按钮	-	打开新增表单弹窗
2	选择菜单类型为"目录"	菜单类型: 目录	显示相关表单项
3	填写目录信息	菜单名称: 系统管理 菜单路径: /system 菜单图标: ze-setting	表单验证通过
4	提交表单	-	1. 保存成功 2. 刷新列表 3. 显示新增节点

表 6-1-12 菜单编辑功能测试

用例编号	Menu003	功能模块	菜单模块
测试目的	验证菜单信息修改功能		
前置条件	存在可编辑的菜单数据		
步骤	操作描述	数据	期望结果
1	点击编辑按钮	-	显示编辑表单弹窗, 数据正确回显
2	修改菜单信息	菜单名称: 新系统管理	表单内容更新
3	提交修改	-	1. 保存成功 2. 列表数据更新 3. 关闭弹窗

表 6-1-13 角色列表功能测试

用例编号	Role001	功能模块	角色模块
测试目的	验证角色列表的基本展示功能		
前置条件	用户已登录且具有角色管理权限		
步骤	操作描述	数据	期望结果
1	访问角色管理页面	-	显示角色列表
2	检查表格列信息	-	显示：角色名称、权限字符、显示顺序、状态、创建时间
3	点击列显示/隐藏按钮	-	显示列配置面板

表 6-1-2-14 角色新增功能测试

用例编号	Role002	功能模块	角色模块
测试目的	验证新增角色功能		
前置条件	已打开角色管理页面		
步骤	操作描述	数据	期望结果
1	点击新增按钮		弹出新增表单
2	填写角色信息	角色名称：测试角色 角色标志：test	表单显示输入内容
3	配置菜单权限	选择菜单	树形菜单正确选中
4	提交表单	-	1. 保存成功 2. 刷新列表 3. 显示成功提示 4. 刷新列表

表 6-1-15 角色编辑测试

用例编号	Role003	功能模块	角色模块
测试目的	验证角色权限配置功能		
前置条件	存在可编辑的角色		
步骤	操作描述	数据	期望结果
1	点击编辑按钮		显示编辑表单
2	展开菜单树	选择菜单	树形菜单正确选中或取消
3	提交表单	-	1. 保存成功 2. 刷新列表 3. 显示成功提示 4. 刷新列表

表 6-1-16 数据权限配置测试

用例编号	Role004	功能模块	角色模块
测试目的	验证角色数据权限配置功能		
前置条件	用户具有数据权限配置权限		
步骤	操作描述	数据	期望结果
1	点击"数据权限"按钮	-	弹出数据权限配置窗口
2	选择权限范围	自定义数据权限	显示部门树选择器
3	配置部门权限	选择对应部门	部门树正确选中
4	切换父子联动	-	父子节点联动生效
4	保存配置	-	1. 保存成功 2. 刷新列表 3. 显示成功提示 4. 刷新列表

表 6-1-17 角色状态切换测试

用例编号	Role005	功能模块	角色模块
测试目的	验证角色状态切换的确认和回滚功能		
前置条件	角色列表包含可操作角色		
步骤	操作描述	数据	期望结果
1	点击状态开关	-	显示确认弹窗
2	取消确认	-	1. 状态回滚 2. 开关恢复原状态
3	确认修改	-	1. 状态更新成功 2. 显示成功提示

6.1.3 单元测试

单元测试，这是一种通常由后端开发人员执行的测试活动，它主要关注于软件系统中各个独立模块的功能性验证。通过精心设计和编写一系列的单元测试用例，开发人员能够确保每一个模块都能够按照既定的预期正常地执行其功能。这种方法论的核心在于，它允许开发团队在进行功能迭代和增加新特性时，能够在一个相对稳定和可控的环境中

工作，从而最大限度地减少对现有逻辑和系统稳定性的负面影响。单元测试的实施有助于提前发现潜在的错误和问题，提高代码质量，并且在长远来看，能够节省维护成本和时间，因为它使得回归测试变得更加容易和高效。具体可查看附件 `zero-admin-api/zero-admin/src/test/java/com/zero/admin/test` 文件夹中查看。

6.1.4 用 Playwright 实现端到端测试

我们可以借助 Playwright^[10]覆盖上面的测试用例，下面以测试用例编号 Auth001 介绍具体使用：

```
test.describe('认证模块: ', () => {  
    test.beforeEach(async ({ page }) => { await page.goto('/login') /* 导航到登录页面*/ })  
    const selector = {  
        username: 'input[prop="username"]',  
        password: 'input[prop="password"]',  
        login_button: 'button[prop="submit"]',  
    }  
    test('Auth001', async ({ page }) => {  
        await page.fill(selector.username, 'admin') // 1. 输入用户名  
        await page.fill(selector.password, 'admin123') // 2. 输入密码  
        const captcha = await ask('请输入验证码: ') // 3. 输入验证码  
        await page.fill('input[prop="captcha"]', captcha)  
        await page.click(selector.login_button) // 4. 点击登录按钮  
        await expect(page).toHaveURL('/dashboard') // 登录成功后成功跳转到首页  
    })  
})
```

本项目端到端测试用例放置在前端仓库 `tests/e2e/` 目录下，通过在控制台运行：“`pnpm test:e2e`”即实际执行指令：`playwright test`，最终我们可以得到如（图 6-1-4）所示一份精美且详尽的自动化测试报告。

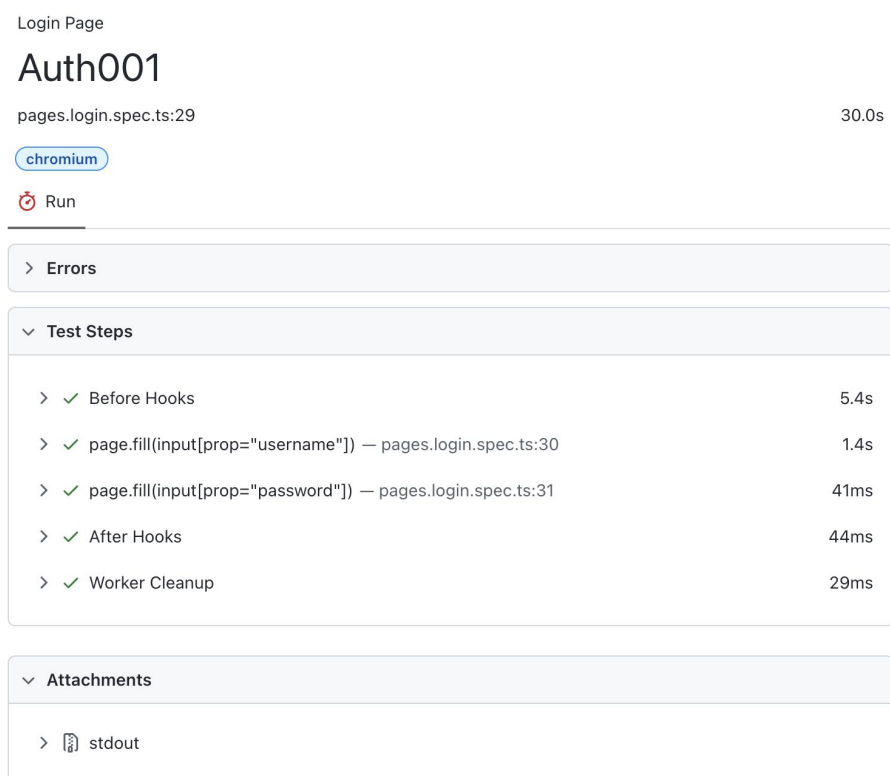


图 6-1-4 Playwright 测试报告

6.2 运行与部署

6.2.1 前端运行与部署

首先我们介绍一下前端运行和构建，首先本企业信息管理系统运行环境要求是：Node.js 版本大于 18.x，pnpm 包管理工具版本大于 8.x，代码仓库使用 Git。

本地开发我们在满足基本运行要求之后，首先使用 `pnpm install` 指令安装 `package.json` 中 `dependencies` 和 `devDependencies` 制定版本的相关依赖，然后确定后端接口网关访问地址，并在 `vite.config` 中进行修改，如：`createProxy('http://localhost:8080/', '/api', /^\/api/)`，表示通过本机 8080 端口并以 `/api` 开头的 url 则代理到本机 8080 指向的后端服务。最后我们运行 `pnpm dev` 启动开发服务器。

生产环境部署我们可以根据生产环境或者测试环境部署，对应使用 `pnpm build:[prod | test]` 进行构建并对打包位置进行区分避免人工发错环境的尴尬局面，与此同时还可对系统已进行构建优化配置，包括：核心库代码分割（Vue、Vue Router、Pinia）、按需加

载组件、静态资源 CDN 优化、图片压缩和缓存策略等，具体如下 vite.config 对应配置：

```
build: {  
  // 生产环境删除 console.log 避免信息泄露以及内存泄漏  
  terserOptions: { compress: { drop_console: true, drop_debugger: true } },  
  rollupOptions: {  
    output: {  
      // 使用合理分包策略，保证更新时客户端资源最小获取  
      manualChunks: { 'index-core': ['vue', 'vue-router', 'pinia', '@vueuse/core'] },  
      chunkFileNames: 'js/[name]-[hash].js', // 引入文件名的名称  
      entryFileNames: 'js/[name]-[hash].js', // 包的入口文件名称  
      assetFileNames: '[ext]/[name]-[hash].[ext]', // 资源文件像 字体，图片等  
    },  
  },  
  // 构建分包，区分生产、测试和开发环境  
  outDir: `dist/${{ production: 'prod', development: 'dev' }[mode] || mode}`,  
},
```

最后我们将存放在 dist/[prod | test]目录的打包结果，将其压缩并解压到服务器指定目录下。

6.2.2 部署环境搭建

本系统以阿里云 ECS 为例来搭建本企业信息管理系统的环境，首先在阿里云官网购买服务器后我们可以制定操作系统，这里我们选择 CentOS 稳定版本进行初始化，然后我们需要依次进行如下应用安装：

A) 安装 Nginx，使用 `yum install nginx` 安装即可。然后对其进行配置，需要配置前端资源对应访问目录，以及制定/api 反向代理到后端 8080 端口的应用服务，我们还可以增加一台 8081 备份服务确保交替部署保证无感更新以及增加系统的稳定性即可以实现负载均衡，配置片段如下：

```

server {
    location / {
        root    /usr/share/nginx/html; # 前端资源存放目录
        try_files $uri $uri/ /index.html; # 支持 vue router history 模式
        index   index.html index.htm;
    }
    location /api/ { proxy_pass http://server/; } # 访问/api 即表示调用接口
}

upstream server {
    ip_hash; # 使用 ip_hash 算法实现负载均衡
    server 127.0.0.1:8080; # 服务器 1
    server 127.0.0.1:8081; # 备份服务器
}

```

B) 安装 redis 缓存数据库，同样运行 `yum install redis` 安装稳定版本即可，然后我们也需要对安全性，访问目录，以及相关策略进行配置：

```

requirepass ruoyi123 # redis 密码
dir /redis/data # 配置持久化文件存储路径
# 配置 rdb
save 900 1 # 15 分钟内有至少 1 个 key 被更改则进行快照
save 300 10 # 5 分钟内有至少 10 个 key 被更改则进行快照
save 60 10000 # 1 分钟内有至少 10000 个 key 被更改则进行快照
rdbcompression yes # 开启压缩
dbfilename dump.rdb # rdb 文件名 用默认的即可

```

C) 安装 Postgresql 数据库，照样是先运行 `yum install postgresql` 安装稳定版本，然后我们需要创建数据库用户、数据库 `zero-admin`，最后我们需要运行数据库建表以及初始化语句，具体可以参见附件 `zero-admin-api/config/init.sql`。

6.2.3 使用 Docker、Github Actions 和阿里云实现后端 CI

在企业级系统开发过程中，部署过程往往非常耗费资源。因此在现代软件开发中，持续集成^[11] (CI) 是一种重要的实践，它允许开发团队频繁地集成代码变更，并及早发现问题。本系统通过结合使用 Docker、GitHub Actions 和阿里云私有镜像，开发人员可以创建一个强大的 CI 流水线，自动化构建、测试和部署过程。Docker 是一个平台，它允许开发人员将应用程序及其依赖项打包到容器中，从而确保在不同环境中的一致性。容器是轻量级的、可移植的，并且可以在任何支持 Docker 的系统上运行，使其成为 CI 流水线的理想选择。GitHub Actions 是一个强大的自动化工具，集成在 GitHub 中，允许开发人员创建自定义工作流，这些工作流可以由代码推送或拉取请求等事件触发。这些工作流可以包括各种步骤，例如构建 Docker 镜像、运行测试和部署应用程序。阿里云私有镜像 提供了一个安全且可扩展的容器镜像仓库服务，允许团队存储和管理 Docker 镜像。通过使用私有镜像，组织可以控制对其镜像的访问，确保只有授权用户可以拉取或推送镜像。要使用 Docker、GitHub Actions 和阿里云私有镜像实现 CI，下面依次介绍本系统实现步骤和过程：

第一步，创建 Dockerfile：在 Dockerfile 中定义应用程序的环境和依赖项。这个文件将用于构建 Docker 镜像。

```
FROM openjdk:21 // 依赖 java 21
LABEL maintainer="Akai"
RUN mkdir -p /zero-admin/server/logs /zero-admin/server/temp //创建必要目录
ENV SPRING_PROFILES_ACTIVE=dev
ADD ./target/zero-admin-web.jar ./app.jar
EXPOSE 8080 // 端口
CMD ["java", "-jar", "./app.jar"] // 打包命令
```

第二步，配置密钥：在 GitHub Secrets 中存储敏感信息，例如阿里云凭证。这确保了凭证不会在工作流文件中暴露。

第三步，设置 GitHub Actions 工作流：创建一个工作流文件（文件位置：.github/workflows/docker-image.yml），定义 CI 流水线。这个文件指定了构建 Docker 镜像、运行测试和将镜像推送到阿里云私有镜像的步骤。

```
name: Backend Docker Image CI

on:
  push:
    branches: [ "release/*" ]
  pull_request:
    branches: [ "release/*" ]

env:
  # 阿里云私有镜像地址:
  ALI_DOCKER_HOST: crpi-3i3pv7x8w1lizxgy.cn-shenzhen.personal.cr.aliyuncs.com

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - name: Set up JDK 21 # 使用 JDK21
        uses: actions/setup-java@v4
        with:
          java-version: '21'
          distribution: 'temurin'

      - name: Build with Maven # 使用 Maven 构建
        run: mvn clean package -D maven.test.skip=true -P prod

      - name: Extract version from branch name # 读取 release/v*.*.* 中的版本号
```



```

    id: extract_version

    run: echo "VERSION=$(echo ${GITHUB_REF#refs/heads/release/})" >>
$GITHUB_ENV

- name: Log in to Ali Docker Hub # 登录阿里镜像仓库
  uses: aliyun/acr-login@v1
  with:
    login-server: ${ env.ALI_DOCKER_HOST }
    username: ${ secrets.ALI_DOCKER_USERNAME }
    password: ${ secrets.ALI_DOCKER_PASSWORD }

- name: Build and push Docker image
  env:
    DOCKER_REPO: ${ env.ALI_DOCKER_HOST }/akai6/zero-admin-api
    IMAGE_TAG: ${ env.VERSION }
  run: | # 使用 docker 构建，并上传到阿里云私有镜像
    docker build . --file zero-admin/Dockerfile --tag
$DOCKER_REPO:$IMAGE_TAG
    docker push $DOCKER_REPO:$IMAGE_TAG

```

第四步，在一般企业开发流程中，都会按照一定 Git 分支管理规范，如：main 分支一般为程序经过测试确定上线后的分支，feature/**是程序某个迭代的开发分支，release/v*.*.*为程序某个版本的预上线分支（常用于测试环节）。本系统 CI 将在代码提交至 release/*分支或初次创建时被触发。

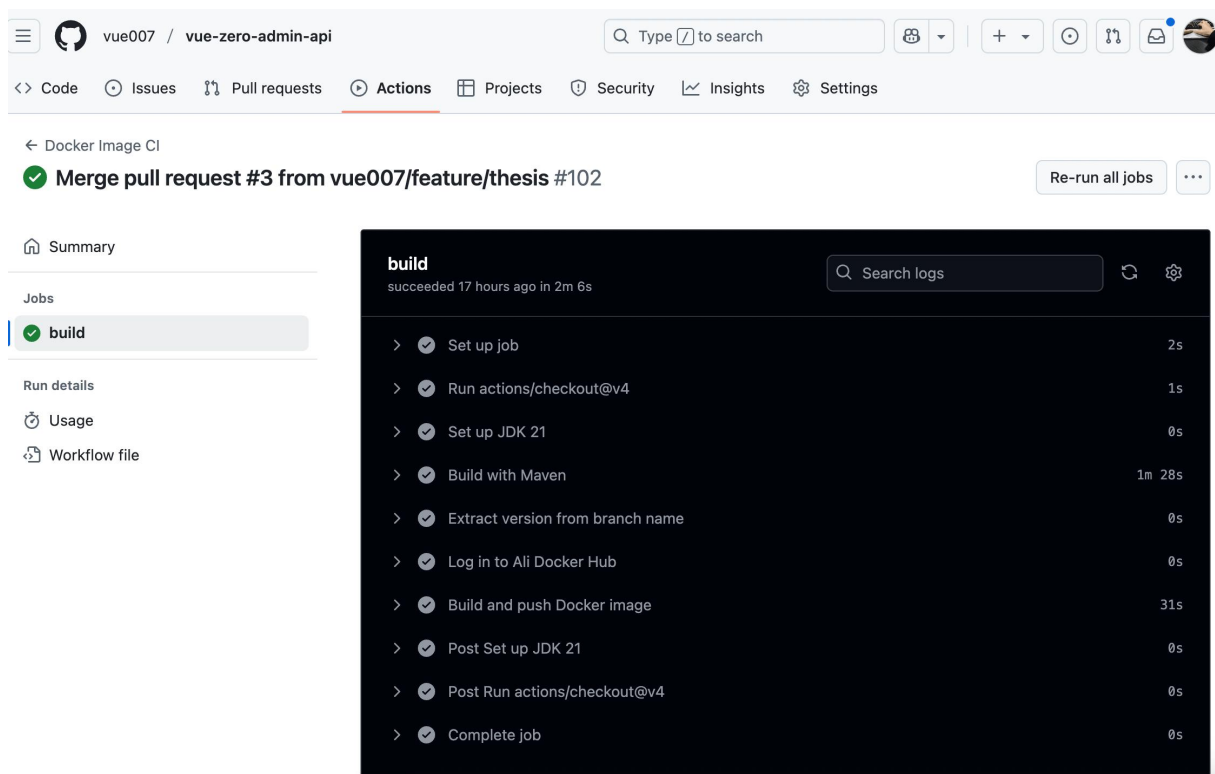


图 6-2-3 GitHub Actions 运行结果

第五步，确定构建成功后如上（图 6-2-3）所示，我们可以登录到阿里云服务器拉取该 docker image 然后 docker run 运行该镜像。

第六步，当程序运行完成后我们可以访问 zero-admin.me 预览本项目示例。

结 论

本文探讨了如何开发一个满足现代企业管理需求的信息管理系统。研究背景指出，随着互联网技术、云计算、大数据和人工智能的发展，企业管理系统在现代企业运营中的重要性日益凸显。但现有的企业管理系统在功能完备性、可扩展性和用户体验等方面存在不足，难以适应企业日益复杂和多样化的管理需求。

本文提出了企业信息管理系统的核心需求。系统需要提供一个现代化的企业信息管理系统模板，涵盖用户管理、租户管理、部门管理、职位管理、菜单管理、角色管理、国际化和字典管理等核心功能，同时支持灵活扩展以满足更多定制需求。

在功能和性能需求分析中，文章重点探讨了三种权限管理模型：ACL、ABAC 和 RBAC。经过比较，RBAC 因其灵活性和效率而成为企业信息管理系统的首选。RBAC 将权限绑定到角色，再将角色分配给用户，实现了细粒度的权限管理，确保数据安全，简化了权限配置过程，完全符合企业的实际需求。其他重要的功能需求包括多租户模式、国际化和用户界面设计。多租户模式允许多个租户共享同一个系统实例，节省资源的同时确保数据隔离，非常适合集团企业或 SaaS 平台。国际化功能通过多语言支持和本地化格式，使系统能够轻松处理跨境业务需求，帮助企业实现全球化。自适应和可主题化的用户界面设计确保系统能在不同设备上提供最佳性能，并允许用户切换主题。

文章还介绍了技术实现的细节，包括使用 Maven 设计模块化的后端系统、利用 Vue 3 组件和 hooks 封装通用的 CRUD 功能，以及使用 CSS 变量和 Sass 预处理器实现响应式和可主题化的用户界面。这些策略特别适合企业级管理系统的快速开发和迭代。

最后，详细介绍了企业开发部署过程，包括代码管理、构建、测试和部署的各个环节。还一步一步地介绍了如何利用 Docker 容器化应用、使用云服务器进行部署、通过 GitHub Actions 自动化工作流实现从代码提交到生产环境的全流程管理，最终实现一个高效、可靠的持续集成和持续交付的系统。

总的来说，本文提出了一种全面的方法来开发满足现代企业需求的信息管理系统，重点关注功能、性能、用户体验和技术实现。该系统旨在解决传统企业管理系统的缺陷，为企业提供一个高效、灵活和易用的管理平台。

参考文献

- [1] 王博.浅谈计算机技术在企业信息化管理中的应用[J].商场现代化, 2023, (11):106-108.DOI:10.14013/j.cnki.scxdh.2023.11.003.
- [2] 贾文强, 刘新, 傅鹏.基于 Spring Boot+Vue 框架的企业记录管理系统设计与实现[J].工业控制计算机, 2024, 37(10):151-152.
- [3] 过晓娇, 田钟晓.一种基于 SaaS 平台的轻量级多租户数据存储模式设计与实现[J].电脑编程技巧与维护, 2023, (05):80-82+143.DOI:10.16184/j.cnki.comprg.2023.05.044.
- [4] 何玉婷.基于浏览器和服务端架构模式的信息管理系统设计研究[J].信息与电脑(理论版), 2021,33(10):159-162.
- [5] Salunke V S ,Ouda A .A Performance Benchmark for the PostgreSQL and MySQL Databases[J].Future Internet,2024,16(10):382-382.
- [6] Nigel P .Docker Deep Dive:Zero to Docker in a single book[M].Packt Publishing Limited:2025-01-27.DOI:10.0000/9781837028344.
- [7] 陈海峰, 丘美玲.基于 RBAC 模型的前后端分离系统设计与实现[J].科技创新与应用, 2024,14(04):102-105+109.DOI:10.19981/j.CN23-1581/G3.2024.04.023.
- [8] 卢万有.基于 JWT 的 RBAC 在前后端分离项目中的设计与实现[J].电脑编程技巧与维护, 2025,(01):46-48.DOI:10.16184/j.cnki.comprg.2025.01.004.
- [9] 吴永娜, 杨春旭, 许佳南.基于 html5+css3 的网页自适应布局设计[J].电脑知识与技术, 2019,15(28):242-244.DOI:10.14004/j.cnki.ckt.2019.3625.
- [10] Microsoft. Playwright Documentation: Introduction[EB/OL]. (2023)[2025-03-27]. <https://playwright.dev/docs/intro>.
- [11] 屠趁锋.基于 DevOps 的持续集成与持续交付流程研究[J].无线互联科技, 2024,21(24):111-114.

致 谢

在论文完成之际，我谨向所有给予我帮助的老师、同学和家人致以最诚挚的谢意。首先，我要衷心感谢我的指导老师陈翠松教授。从论文选题到最终定稿，陈老师始终以渊博的学识、严谨的治学态度和精益求精的工作作风给予我全方位指导。每当研究遇到瓶颈时，陈老师总能以独到的学术见解为我指明方向，这种春风化雨般的教诲令我终身受益。同时，特别感谢陈翠松老师在论文撰写过程中给予的鼎力支持。无论是研究框架的构建、文献资料的梳理，还是研究方法的选用，陈老师都不吝赐教。他深厚的学术造诣与诲人不倦的师者风范，为我的学术成长奠定了坚实基础。此外，还要感谢各位评审专家提出的宝贵意见，以及同窗好友们在研究过程中的热心帮助。最后，谨以此文献给我的家人，感谢他们始终如一地理解与支持。